



Anjana Infotech

Your Dream We Display

An ISO 9001 : 2015 Certified Company

PYTHON PROGRAMMING DOCUMENTATION

An Easy and Complete Guide to Understanding Python Programming Concepts

This document is designed to provide clear and simple explanations of all Python programming concepts. Each topic is explained in an easy-to-understand manner with real-time examples, proper syntax, sample code, and output explanations.

The purpose of this documentation is to help beginners as well as learners to understand Python in a practical way and apply it in real-world applications.

Date: March 2026

Organization: Anjana Infotech

TABLE OF CONTENTS

#	Topic	Page
1	What is Python?	1
2	Installing Python	2
3	Your First Program	3
4	Variables	4
5	Data Types	5
6	Numbers	6
7	Strings	7
8	String Methods	8
9	Taking User Input	9
10	Type Conversion	10
11	Comments	11
12	Lists	12
13	List Methods	13
14	Tuples	14
15	Sets	15
16	Dictionaries	16
17	Dictionary Methods	17

#	Topic	Page
18	if Statements	18
19	if-elif-else	19
20	Nested Conditions	20
21	for Loops	21
22	for Loops with range()	22
23	while Loops	23
24	break Statement	24
25	continue Statement	25
26	Functions	26
27	Function Parameters	27
28	Return Values	28
29	Default Arguments	29
30	Keyword Arguments	30
31	Variable Scope	31
32	Lambda Functions	32
33	map() and filter()	33
34	List Comprehensions	34

#	Topic	Page
35	Dictionary Comprehensions	35
36	Modules	36
37	Creating Modules	37
38	File Reading	38
39	File Writing	39
40	Exception Handling	40
41	try-except-else-finally	41
42	Classes and Objects	42
43	Class Constructor (init)	43
44	Inheritance	44
45	Multiple Inheritance	45
46	Encapsulation	46
47	Polymorphism	47
48	Iterators	48
49	Generators	49
50	Decorators	50
51	Date and Time	51

#	Topic	Page
52	Regular Expressions	52
53	JSON Handling	53
54	Virtual Environments	54
55	pip and Packages	55
56	NumPy Basics	56
57	Pandas Basics	57
58	Matplotlib Basics	58
59	Web Scraping Basics	59
60	API Requests	60

TOPIC 1: WHAT IS PYTHON?

Clear Explanation

Python is a high-level, interpreted programming language created by Guido van Rossum in 1991. It is designed to be easy to read and write. Python uses indentation instead of curly braces to define code blocks, making it very clean and readable.

Key Features:

- Easy to learn and use
- Interpreted language (no compilation needed)
- Object-oriented
- Large standard library
- Cross-platform

Real-Time Example

Think of Python as a Swiss Army knife. You can use it for:

- Building websites (Instagram, Spotify)

- Data analysis (Netflix recommendations)
- Artificial Intelligence (ChatGPT uses Python)
- Automation (sending emails automatically)
- Game development

Sample Code

```
python
```

```
# This is a simple Python program
```

```
print("Python is awesome!")
```

```
print("Learning is fun!")
```

Output

text

Python is awesome!

Learning is fun!

Explanation

- `print()` is a built-in function that displays output
- The `#` symbol is used for comments
- Python executes code line by line from top to bottom

TOPIC 2: INSTALLING PYTHON

Clear Explanation

Before writing Python code, you need to install Python on your computer. You can download the latest version from python.org. After installation, you can verify it's working by opening Command Prompt (Windows) or Terminal (Mac/Linux) and typing `python --version`.

Real-Time Example

Installing Python is like installing WhatsApp on your phone. Once installed, you can start using it immediately.

Installation Steps:

1. Go to python.org/downloads
2. Download the installer for your operating system
3. Run the installer
4. **IMPORTANT:** Check "Add Python to PATH" (Windows)
5. Click Install

Verification Commands

bash

For Windows

python --version

For Mac/Linux

python3 --version

Sample Code (Test Installation)

Create a file called test.py and add:

```
python
```

```
print("Python is installed successfully!")
```

Output (in terminal)

text

Python 3.12.0

Python is installed successfully!

Explanation

- --version shows which Python version is installed
- If you see a version number, installation was successful

TOPIC 3: YOUR FIRST PROGRAM

Clear Explanation

"Hello, World!" is the traditional first program when learning any programming language. It confirms that everything is set up correctly and teaches you the basic syntax of printing output.

Real-Time Example

When you buy a new TV, you turn it on to check if it works. "Hello, World!" is your way of checking if Python works.

Two Ways to Run Python:

1. **Interactive Mode:** Type python in terminal and write code directly
2. **Script Mode:** Create a .py file and run it

Sample Code

Create a file named hello.py:

```
python
```

```
# My first Python program  
print("Hello, World!")  
print("Welcome to Python programming!")  
print("Let's learn together!")
```

Output

text

Hello, World!

Welcome to Python programming!

Let's learn together!

How to Run:

bash

python hello.py

or

python3 hello.py

Explanation of the Code

- Line 1: # starts a comment - ignored by Python
- Line 2-4: print() displays text on screen
- Each print() adds a new line automatically

TOPIC 4: VARIABLES

Clear Explanation

Variables are containers that store data. In Python, you don't need to declare what type of data a variable will hold. Python figures it out automatically. Variables are created when you assign a value to them.

Real-Time Example

Variables are like labeled storage boxes:

- Box labeled "age" contains the number 25
- Box labeled "name" contains the text "John"
- You can open the box anytime to see what's inside

Rules for Variable Names:

- Can contain letters, numbers, and underscores
- Must start with a letter or underscore

- Cannot start with a number
- Cannot use Python keywords (if, else, for, etc.)
- Case-sensitive (Age and age are different)

Sample Code

```
python
```

```
# Valid variable names
```

```
name = "Alice"
```

```
age = 25
```

```
height = 5.6
```

```
is_student = True
```

```
score = 98.5
```

```
city_name = "New York"
```

```
# Print variables
```

```
print(name)
```

```
print(age)
```

```
print(height)
```

```
print(is_student)
```

```
print(score)
```

```
print(city_name)
```

```
# Variables can change
```

```
age = 26 # age now becomes 26
```

```
print("Updated age:", age)
```

Output

```
text
```

```
Alice
```

```
25
```

```
5.6
```

```
True
```

```
98.5
```

New York

Updated age: 26

Explanation

- Variables store values for later use
 - You can change a variable's value by assigning a new one
 - Variable names should be descriptive (use `student_name` not just `n`)
-

TOPIC 5: DATA TYPES

Clear Explanation

Data types specify what kind of data a variable can hold. Python has several built-in data types. The most common are:

- **int**: Integer numbers (whole numbers)
- **float**: Decimal numbers
- **str**: Strings (text)
- **bool**: Boolean (True/False)
- **list**: Ordered, changeable collection
- **tuple**: Ordered, unchangeable collection
- **dict**: Key-value pairs
- **set**: Unordered, unique items

Real-Time Example

Different data types are like different kitchen tools:

- Knife (`int`) - for cutting numbers
- Measuring cup (`float`) - for precise measurements
- Recipe book (`str`) - for text instructions
- On/Off switch (`bool`) - for True/False states

Sample Code

```
python
```

```
# Different data types
```

```
age = 30          # int
```

```
price = 19.99     # float
```

```
name = "Python"   # str
```

```
is_valid = True      # bool
fruits = ["apple", "banana", "orange"] # list
coordinates = (10, 20) # tuple
person = {"name": "John", "age": 25}    # dict
unique_numbers = {1, 2, 3, 4, 5}        # set
```

Check data types

```
print(type(age))
print(type(price))
print(type(name))
print(type(is_valid))
print(type(fruits))
print(type(coordinates))
print(type(person))
print(type(unique_numbers))
```

Output

text

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'list'>
<class 'tuple'>
<class 'dict'>
<class 'set'>
```

Explanation

- Use type() function to check data type
- Python automatically assigns data type based on value
- Different types have different properties and methods

TOPIC 6: NUMBERS

Clear Explanation

Python supports two main number types: int (integers) and float (floating-point numbers). You can perform various arithmetic operations on numbers.

Real-Time Example

Numbers are everywhere in real life:

- Counting money (int)
- Measuring ingredients (float)
- Calculating discounts
- Tracking scores in games

Arithmetic Operators:

Operator	Meaning	Example
+	Addition	$5 + 3 = 8$
-	Subtraction	$5 - 3 = 2$
*	Multiplication	$5 * 3 = 15$
/	Division	$5 / 2 = 2.5$
//	Floor Division	$5 // 2 = 2$
%	Modulus (remainder)	$5 \% 2 = 1$
**	Exponentiation	$5 ** 2 = 25$

Sample Code

```
python
# Basic arithmetic
a = 15
b = 4

print("Addition:", a + b)
print("Subtraction:", a - b)
```

```
print("Multiplication:", a * b)
print("Division:", a / b)
print("Floor Division:", a // b)
print("Modulus:", a % b)
print("Exponent:", a ** b)
```

Working with floats

```
price = 19.99
tax = 0.08
total = price + (price * tax)
print("Total with tax:", total)
```

Type checking

```
print(type(a))
print(type(price))
```

Output

text

Addition: 19

Subtraction: 11

Multiplication: 60

Division: 3.75

Floor Division: 3

Modulus: 3

Exponent: 50625

Total with tax: 21.5892

<class 'int'>

<class 'float'>

Explanation

- / always returns a float even if result is a whole number
- // returns an integer (rounds down)
- % is useful for checking if a number is even/odd

- You can combine operations: $\text{result} = (a + b) * c$
-

TOPIC 7: STRINGS

Clear Explanation

Strings are sequences of characters enclosed in quotes. You can use single quotes (') or double quotes ("). Strings are immutable (cannot be changed after creation).

Real-Time Example

Strings are like a sentence written on a whiteboard:

- You can read each letter
- You can copy parts of it
- But you can't erase or change individual letters (you must rewrite the whole sentence)

String Operations:

```
python
```

```
# Creating strings
```

```
name = "Alice"
```

```
message = 'Hello World'
```

```
long_text = """This is a  
multi-line string"""
```

```
# String indexing (starts at 0)
```

```
word = "Python"
```

```
print(word[0]) # P
```

```
print(word[1]) # y
```

```
print(word[-1]) # n (last character)
```

Sample Code

```
python
```

```
# Different ways to create strings
```

```
single = 'Hello'
```

```
double = "World"
```

```
multi = """This is
```

```
a multiline
```

```
string"""
```

```
print(single, double)
```

```
print(multi)
```

```
# String concatenation
```

```
first_name = "John"
```

```
last_name = "Doe"
```

```
full_name = first_name + " " + last_name
```

```
print(full_name)
```

```
# String repetition
```

```
print("Ha" * 3) # HaHaHa
```

```
# String length
```

```
text = "Python Programming"
```

```
print("Length:", len(text))
```

```
# Accessing characters
```

```
word = "Computer"
```

```
print("First character:", word[0])
```

```
print("Last character:", word[-1])
```

```
print("Characters 2-5:", word[1:5])
```

Output

```
text
```

```
Hello World
```

```
This is
```

```
a multiline
```

```
string
```

```
John Doe
```

```
HaHaHa
```

Length: 19

First character: C

Last character: r

Characters 2-5: ompu

Explanation

- Strings can be created with single, double, or triple quotes
- + joins strings together
- * repeats strings
- len() returns the number of characters
- String indexing: [start:end] extracts characters from start to end-1

TOPIC 8: STRING METHODS

Clear Explanation

Python provides many built-in methods to manipulate strings. These methods help you change case, find text, replace content, remove spaces, and more.

Real-Time Example

String methods are like text editing tools:

- upper() = CAPS LOCK key
- lower() = lowercase text
- strip() = removing extra spaces
- replace() = find and replace

Common String Methods:

Method	Description	Example
upper()	Converts to uppercase	"hello".upper() → "HELLO"
lower()	Converts to lowercase	"HELLO".lower() → "hello"
strip()	Removes leading/trailing spaces	" hello ".strip() → "hello"
replace()	Replaces text	"hi".replace("h","b") → "bi"

Method	Description	Example
<code>split()</code>	Splits into list	<code>"a,b".split(",") → ["a","b"]</code>
<code>find()</code>	Finds position	<code>"hello".find("e") → 1</code>
<code>count()</code>	Counts occurrences	<code>"hello".count("l") → 2</code>

Sample Code

python

```
text = " Python Programming is Fun! "
```

Case conversion

```
print("Upper:", text.upper())
```

```
print("Lower:", text.lower())
```

```
print("Title:", text.title())
```

Removing whitespace

```
print("Strip:", text.strip())
```

```
print("Left strip:", text.lstrip())
```

```
print("Right strip:", text.rstrip())
```

Searching and replacing

```
print("Replace:", text.replace("Fun", "Awesome"))
```

```
print("Find 'Programming':", text.find("Programming"))
```

```
print("Count 'P':", text.count("P"))
```

Splitting

```
words = text.strip().split()
```

```
print("Split into words:", words)
```

Checking string content

```
email = "user@example.com"

print("Starts with 'user':", email.startswith("user"))

print("Ends with '.com':", email.endswith(".com"))

print("Is alphabetic?", "Python123".isalpha())

print("Is alphanumeric?", "Python123".isalnum())
```

Output

text

Upper: PYTHON PROGRAMMING IS FUN!

Lower: python programming is fun!

Title: Python Programming Is Fun!

Strip: Python Programming is Fun!

Left strip: Python Programming is Fun!

Right strip: Python Programming is Fun!

Replace: Python Programming is Awesome!

Find 'Programming': 8

Count 'P': 2

Split into words: ['Python', 'Programming', 'is', 'Fun!']

Starts with 'user': True

Ends with '.com': True

Is alphabetic? False

Is alphanumeric? True

Explanation

- String methods return new strings (don't modify original)
- strip() is useful for cleaning user input
- split() converts strings to lists for processing
- find() returns -1 if text not found

TOPIC 9: TAKING USER INPUT

Clear Explanation

The input() function allows your program to get input from the user. The function always returns a string, even if the user enters numbers.

Real-Time Example

Like a cashier asking "What would you like to order?" and waiting for your response.

Sample Code

```
python
```

```
# Basic input
```

```
name = input("What is your name? ")
```

```
print("Hello,", name)
```

```
# Taking numbers (need conversion)
```

```
age = input("How old are you? ")
```

```
# age is a string here, so we convert to int
```

```
age = int(age)
```

```
print("Next year you will be", age + 1)
```

```
# Taking multiple inputs
```

```
print("\n--- Calculator ---")
```

```
num1 = float(input("Enter first number: "))
```

```
num2 = float(input("Enter second number: "))
```

```
result = num1 + num2
```

```
print(num1, "+", num2, "=", result)
```

Output (Example)

```
text
```

```
What is your name? Alice
```

```
Hello, Alice
```

```
How old are you? 25
```

```
Next year you will be 26
```

```
--- Calculator ---
```

```
Enter first number: 10.5
```

```
Enter second number: 5.2
```

```
10.5 + 5.2 = 15.7
```

Explanation

- `input()` displays a prompt and waits for user input
 - The user's input is always returned as a string
 - Use `int()` or `float()` to convert to numbers
 - Always validate user input in real applications
-

TOPIC 10: TYPE CONVERSION

Clear Explanation

Type conversion (type casting) is converting one data type to another. Python does some conversions automatically (implicit), but sometimes you need to do it manually (explicit).

Real-Time Example

Imagine you have water (liquid) and you freeze it to make ice (solid). Same substance, different form. Type conversion is similar - same value, different type.

Sample Code

python

Implicit conversion (automatic)

`x = 10    # int`

`y = 3.5    # float`

`z = x + y    # int + float = float`

`print("Implicit conversion:", z, type(z))`

Explicit conversion (manual)

String to integer

`num_str = "100"`

`num_int = int(num_str)`

`print("String to int:", num_int, type(num_int))`

String to float

`price_str = "19.99"`

`price_float = float(price_str)`

`print("String to float:", price_float, type(price_float))`

Number to string

```
age = 25  
age_str = str(age)  
print("Int to string:", age_str, type(age_str))
```

Float to int (truncates decimal)

```
decimal = 7.89  
integer = int(decimal)  
print("Float to int:", integer)
```

Int to float

```
count = 5  
count_float = float(count)  
print("Int to float:", count_float)
```

Boolean conversions

```
print("bool(0):", bool(0))  # False  
print("bool(1):", bool(1))  # True  
print("bool(''):", bool('')) # False  
print("bool('hi'):", bool("hi")) # True
```

Output

text

Implicit conversion: 13.5 <class 'float'>

String to int: 100 <class 'int'>

String to float: 19.99 <class 'float'>

Int to string: 25 <class 'str'>

Float to int: 7

Int to float: 5.0

bool(0): False

bool(1): True

bool(''): False

`bool('hi'): True`

Explanation

- `int()` converts to integer (truncates decimal)
 - `float()` converts to floating-point number
 - `str()` converts to string
 - `bool()` converts to boolean (0, None, empty = False)
 - Always convert user input from string to appropriate type
-

TOPIC 11: COMMENTS

Clear Explanation

Comments are notes in your code that Python ignores. They help you and others understand what your code does. Good comments explain WHY, not WHAT.

Real-Time Example

Comments are like sticky notes on a recipe:

- "Add flour slowly to avoid lumps" (explains WHY)
- Not "Add flour" (that's obvious from the code)

Types of Comments:

1. Single-line Comments

python

This is a single-line comment

`name = "John" # This is an inline comment`

2. Multi-line Comments

python

"""

This is a multi-line comment

It can span multiple lines

Use triple quotes

"""

3. Docstrings (Documentation Strings)

python

`def add_numbers(a, b):`

```
"""
```

This function adds two numbers and returns the result.

Parameters:

a (int/float): First number

b (int/float): Second number

Returns:

int/float: Sum of a and b

```
"""
```

```
    return a + b
```

Sample Code

```
python
```

```
# Program to calculate student's average grade
```

```
# Author: John Doe
```

```
# Date: 2024-01-15
```

```
# Get three test scores from user
```

```
test1 = float(input("Enter first test score: "))
```

```
test2 = float(input("Enter second test score: "))
```

```
test3 = float(input("Enter third test score: "))
```

```
# Calculate average (sum divided by count)
```

```
average = (test1 + test2 + test3) / 3
```

```
# Display result with grade
```

```
print("Average score:", average)
```

```
# Check grade using if-else
```

```
if average >= 90:
```

```
    print("Grade: A") # Excellent performance
```

```
elif average >= 80:
    print("Grade: B") # Good performance
elif average >= 70:
    print("Grade: C") # Satisfactory
else:
    print("Grade: F") # Needs improvement
```

Output (Example)

text

Enter first test score: 85

Enter second test score: 90

Enter third test score: 78

Average score: 84.33333333333333

Grade: B

Best Practices for Comments

- Comment WHY, not WHAT (code shows what)
- Keep comments up to date
- Use comments for complex logic
- Don't state the obvious
- Use docstrings for functions and classes

TOPIC 12: LISTS

Clear Explanation

Lists are ordered, changeable collections that can hold items of different data types. Lists are created using square brackets [] and items are separated by commas.

Real-Time Example

A list is like a shopping list:

- You can add items (append)
- Remove items (remove)
- Change items (update)
- Items are in a specific order

Sample Code

python

Creating lists

```
fruits = ["apple", "banana", "orange"]
```

```
numbers = [1, 2, 3, 4, 5]
```

```
mixed = [1, "hello", 3.14, True]
```

```
empty = []
```

```
print("Fruits:", fruits)
```

```
print("Numbers:", numbers)
```

```
print("Mixed:", mixed)
```

Accessing list items

```
print("First fruit:", fruits[0])
```

```
print("Last fruit:", fruits[-1])
```

```
print("First two fruits:", fruits[0:2])
```

Changing list items

```
fruits[1] = "mango"
```

```
print("Updated fruits:", fruits)
```

List length

```
print("Number of fruits:", len(fruits))
```

Checking if item exists

```
if "apple" in fruits:
```

```
    print("Apple is in the list")
```

Output

text

```
Fruits: ['apple', 'banana', 'orange']
```

```
Numbers: [1, 2, 3, 4, 5]
```

```
Mixed: [1, 'hello', 3.14, True]
```

First fruit: apple

Last fruit: orange

First two fruits: ['apple', 'banana']

Updated fruits: ['apple', 'mango', 'orange']

Number of fruits: 3

Apple is in the list

Explanation

- Lists maintain order of items
- Indexing starts at 0
- Negative indexing starts from end (-1 is last)
- Lists can contain mixed data types
- Lists are mutable (can be changed)

TOPIC 13: LIST METHODS

Clear Explanation

Lists have many built-in methods for adding, removing, sorting, and manipulating items. These methods modify the original list.

Real-Time Example

List methods are like tools for managing your shopping list:

- `append()` = add item at end
- `insert()` = add item at specific position
- `remove()` = take item out
- `sort()` = organize alphabetically

Common List Methods:

Method	Description	Example
<code>append(x)</code>	Adds x to end	<code>list.append(5)</code>
<code>insert(i,x)</code>	Inserts x at index i	<code>list.insert(0,5)</code>
<code>extend(list)</code>	Adds another list	<code>list.extend([1,2])</code>

Method	Description	Example
<code>remove(x)</code>	Removes first x	<code>list.remove(5)</code>
<code>pop(i)</code>	Removes item at i	<code>list.pop(0)</code>
<code>clear()</code>	Removes all items	<code>list.clear()</code>
<code>index(x)</code>	Returns index of x	<code>list.index(5)</code>
<code>count(x)</code>	Counts occurrences	<code>list.count(5)</code>
<code>sort()</code>	Sorts ascending	<code>list.sort()</code>
<code>reverse()</code>	Reverses order	<code>list.reverse()</code>

Sample Code

python

Create a list

```
shopping_list = ["milk", "bread", "eggs"]
```

```
print("Initial:", shopping_list)
```

Adding items

```
shopping_list.append("butter")
```

```
print("After append:", shopping_list)
```

```
shopping_list.insert(1, "cheese")
```

```
print("After insert:", shopping_list)
```

Adding multiple items

```
shopping_list.extend(["juice", "cereal"])
```

```
print("After extend:", shopping_list)
```

Removing items

```
shopping_list.remove("bread")
print("After remove:", shopping_list)
```

Remove last item

```
last_item = shopping_list.pop()
print("Popped:", last_item)
print("After pop:", shopping_list)
```

Sorting

```
numbers = [3, 1, 4, 1, 5, 9, 2]
numbers.sort()
print("Sorted numbers:", numbers)
```

```
numbers.sort(reverse=True)
print("Descending:", numbers)
```

Counting and finding

```
fruits = ["apple", "banana", "apple", "orange"]
print("Apple count:", fruits.count("apple"))
print("Banana index:", fruits.index("banana"))
```

Copying a list

```
fruits_copy = fruits.copy()
print("Copy:", fruits_copy)
```

Clearing a list

```
fruits_copy.clear()
print("Cleared:", fruits_copy)
```

Output

text

Initial: ['milk', 'bread', 'eggs']

After append: ['milk', 'bread', 'eggs', 'butter']

After insert: ['milk', 'cheese', 'bread', 'eggs', 'butter']

After extend: ['milk', 'cheese', 'bread', 'eggs', 'butter', 'juice', 'cereal']

After remove: ['milk', 'cheese', 'eggs', 'butter', 'juice', 'cereal']

Popped: cereal

After pop: ['milk', 'cheese', 'eggs', 'butter', 'juice']

Sorted numbers: [1, 1, 2, 3, 4, 5, 9]

Descending: [9, 5, 4, 3, 2, 1, 1]

Apple count: 2

Banana index: 1

Copy: ['apple', 'banana', 'apple', 'orange']

Cleared: []

Explanation

- Most methods modify the original list
- `append()` adds one item, `extend()` adds multiple
- `pop()` returns and removes an item
- `sort()` modifies the list in place
- Use `copy()` to create a new list (not just reference)

TOPIC 14: TUPLES

Clear Explanation

Tuples are ordered, unchangeable collections. Once created, you cannot modify a tuple. Tuples are created using parentheses ().

Real-Time Example

A tuple is like a birth certificate:

- Information is fixed (date of birth, place)
- You can read it, but can't change it
- If you need changes, you must create a new one

Sample Code

```
python
```

```
# Creating tuples
```

```
person = ("John", 25, "New York")
coordinates = (40.7128, -74.0060)
single_item = (5,) # Note the comma
empty = ()

print("Person:", person)
print("Coordinates:", coordinates)
print("Single item tuple:", single_item)

# Accessing tuple items
print("Name:", person[0])
print("Age:", person[1])
print("City:", person[2])

# Tuple unpacking
name, age, city = person
print("Unpacked - Name:", name)
print("Unpacked - Age:", age)
print("Unpacked - City:", city)

# Tuples can contain mixed types
mixed = (1, "hello", 3.14, True)
print("Mixed tuple:", mixed)

# Trying to modify (will cause error)
# person[0] = "Jane" # This will raise TypeError

# Tuple methods
print("Count of 25:", person.count(25))
print("Index of 'John':", person.index("John"))
```

Converting list to tuple

```
fruits_list = ["apple", "banana"]  
fruits_tuple = tuple(fruits_list)  
print("Tuple from list:", fruits_tuple)
```

Output

text

Person: ('John', 25, 'New York')

Coordinates: (40.7128, -74.006)

Single item tuple: (5,)

Name: John

Age: 25

City: New York

Unpacked - Name: John

Unpacked - Age: 25

Unpacked - City: New York

Mixed tuple: (1, 'hello', 3.14, True)

Count of 25: 1

Index of 'John': 0

Tuple from list: ('apple', 'banana')

When to Use Tuples

- When data should not change (constants, configurations)
- For dictionary keys (lists can't be keys)
- Better performance than lists for large data
- Returning multiple values from functions

Explanation

- Tuples are immutable (cannot change)
 - Faster than lists for iteration
 - Use parentheses () (though commas create tuples)
 - Single-item tuples need trailing comma
 - Can be unpacked into variables
-

TOPIC 15: SETS

Clear Explanation

Sets are unordered collections of unique items. Sets cannot have duplicate values. They are created using curly braces {} or the set() function.

Real-Time Example

A set is like a bag of unique marbles:

- Each marble appears only once
- Order doesn't matter
- You can add or remove marbles
- You can check if a specific marble exists

Sample Code

```
python
```

```
# Creating sets
```

```
fruits = {"apple", "banana", "orange", "apple"} # Duplicate removed
```

```
numbers = set([1, 2, 3, 4, 5])
```

```
empty = set() # Note: {} creates empty dict
```

```
print("Fruits:", fruits)
```

```
print("Numbers:", numbers)
```

```
# Adding items
```

```
fruits.add("mango")
```

```
print("After add:", fruits)
```

```
# Removing items
```

```
fruits.remove("banana") # Error if item doesn't exist
```

```
fruits.discard("grape") # No error if item doesn't exist
```

```
print("After remove:", fruits)
```

```
# Set operations
```

```
set_a = {1, 2, 3, 4, 5}
```

```

set_b = {4, 5, 6, 7, 8}

print("Union:", set_a | set_b)    # Combine sets
print("Intersection:", set_a & set_b) # Common elements
print("Difference:", set_a - set_b) # In A but not in B
print("Symmetric difference:", set_a ^ set_b) # In either but not both

# Checking membership
if "apple" in fruits:
    print("Apple is in the set")

# Set length
print("Number of fruits:", len(fruits))

# Looping through set
for fruit in fruits:
    print("Fruit:", fruit)

```

Output

text

Fruits: {'orange', 'apple', 'banana'}

Numbers: {1, 2, 3, 4, 5}

After add: {'mango', 'orange', 'apple', 'banana'}

After remove: {'mango', 'orange', 'apple'}

Union: {1, 2, 3, 4, 5, 6, 7, 8}

Intersection: {4, 5}

Difference: {1, 2, 3}

Symmetric difference: {1, 2, 3, 6, 7, 8}

Apple is in the set

Number of fruits: 3

Fruit: mango

Fruit: orange

Fruit: apple

Set Methods

Method	Description
<code>add(x)</code>	Adds item x
<code>remove(x)</code>	Removes x (error if not exists)
<code>discard(x)</code>	Removes x (no error if not exists)
<code>pop()</code>	Removes and returns random item
<code>clear()</code>	Removes all items
<code>union()</code>	Returns union of sets
<code>intersection()</code>	Returns intersection
<code>difference()</code>	Returns difference

Explanation

- Sets automatically remove duplicates
- Unordered - cannot access by index
- Very fast for checking membership (in operator)
- Great for mathematical set operations

TOPIC 16: DICTIONARIES

Clear Explanation

Dictionaries store data in key-value pairs. Each key is unique and is used to access its associated value. Dictionaries are created using curly braces `{}` with colons separating keys and values.

Real-Time Example

A dictionary is like a phone book:

- Name (key) → Phone number (value)
- You look up by name, not by position

- Each name appears only once

Sample Code

python

Creating dictionaries

```
person = {  
    "name": "John",  
    "age": 30,  
    "city": "New York"  
}
```

```
print("Person:", person)
```

Accessing values

```
print("Name:", person["name"])
```

```
print("Age:", person["age"])
```

Using get() method (safer)

```
print("City:", person.get("city"))
```

```
print("Country:", person.get("country", "Not found"))
```

Adding/updating items

```
person["email"] = "john@example.com" # Add new
```

```
person["age"] = 31 # Update existing
```

```
print("Updated:", person)
```

Removing items

```
removed = person.pop("city")
```

```
print("Removed city:", removed)
```

```
print("After pop:", person)
```

Remove last item

```
last = person.popitem()
print("Removed last:", last)
```

Checking if key exists

```
if "name" in person:
    print("Name exists in dictionary")
```

Dictionary length

```
print("Number of items:", len(person))
```

Getting all keys and values

```
print("Keys:", person.keys())
print("Values:", person.values())
print("Items:", person.items())
```

Output

text

Person: {'name': 'John', 'age': 30, 'city': 'New York'}

Name: John

Age: 30

City: New York

Country: Not found

Updated: {'name': 'John', 'age': 31, 'city': 'New York', 'email': 'john@example.com'}

Removed city: New York

After pop: {'name': 'John', 'age': 31, 'email': 'john@example.com'}

Removed last: ('email', 'john@example.com')

Name exists in dictionary

Number of items: 2

Keys: dict_keys(['name', 'age'])

Values: dict_values(['John', 31])

Items: dict_items([('name', 'John'), ('age', 31)])

Explanation

- Keys must be unique and immutable (strings, numbers, tuples)
 - Values can be any data type
 - Dictionaries are unordered (Python 3.7+ maintains insertion order)
 - Fast lookups by key
 - Can be nested (dictionaries within dictionaries)
-

TOPIC 17: DICTIONARY METHODS

Clear Explanation

Dictionaries have many useful methods for accessing, modifying, and iterating through key-value pairs.

Sample Code

```
python
```

```
# Creating a dictionary
```

```
student = {  
    "name": "Alice",  
    "age": 20,  
    "courses": ["Math", "Science"],  
    "grade": "A"  
}
```

```
print("Original:", student)
```

```
# Copy a dictionary
```

```
student_copy = student.copy()  
print("Copy:", student_copy)
```

```
# Update dictionary
```

```
new_info = {"grade": "A+", "year": 2024}  
student.update(new_info)  
print("After update:", student)
```

```
# Set default value if key doesn't exist

student.setdefault("major", "Computer Science")

student.setdefault("age", 25) # Won't change existing

print("After setdefault:", student)
```

```
# Iterating through dictionary

print("\n--- Iterating ---")

for key in student:

    print(f"{key}: {student[key]}")
```

```
# Using items() to get key-value pairs

print("\n--- Using items() ---")

for key, value in student.items():

    print(f"{key}: {value}")
```

```
# Dictionary comprehension

squares = {x: x**2 for x in range(1, 6)}

print("Squares:", squares)
```

```
# Nested dictionaries

users = {

    "user1": {"name": "John", "age": 25},

    "user2": {"name": "Jane", "age": 24}

}

print("Nested dictionary:", users)

print("User1 name:", users["user1"]["name"])
```

Output

text

Original: {'name': 'Alice', 'age': 20, 'courses': ['Math', 'Science'], 'grade': 'A'}

Copy: {'name': 'Alice', 'age': 20, 'courses': ['Math', 'Science'], 'grade': 'A'}

After update: {'name': 'Alice', 'age': 20, 'courses': ['Math', 'Science'], 'grade': 'A+', 'year': 2024}

After setdefault: {'name': 'Alice', 'age': 20, 'courses': ['Math', 'Science'], 'grade': 'A+', 'year': 2024, 'major': 'Computer Science'}

--- Iterating ---

name: Alice

age: 20

courses: ['Math', 'Science']

grade: A+

year: 2024

major: Computer Science

--- Using items() ---

name: Alice

age: 20

courses: ['Math', 'Science']

grade: A+

year: 2024

major: Computer Science

Squares: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Nested dictionary: {'user1': {'name': 'John', 'age': 25}, 'user2': {'name': 'Jane', 'age': 24}}

User1 name: John

Common Dictionary Methods

Method	Description
keys()	Returns all keys
values()	Returns all values
items()	Returns key-value pairs
get(key)	Returns value or None

Method	Description
pop(key)	Removes and returns value
popitem()	Removes last item
update(dict)	Merges dictionary
copy()	Creates shallow copy
clear()	Removes all items
setdefault(key,default)	Sets default if key missing

Explanation

- items() is most useful for iteration
- get() is safer than direct access
- Dictionary comprehensions provide concise creation
- Nested dictionaries are common for complex data

TOPIC 18: IF STATEMENTS

Clear Explanation

The if statement allows your program to make decisions. It executes a block of code only if a condition is True.

Real-Time Example

Like a security guard:

- If ID is valid → let them enter
- If not → deny entry

Comparison Operators:

Operator	Meaning	Example
==	Equal to	5 == 5 → True

Operator	Meaning	Example
!=	Not equal to	5 != 3 → True
>	Greater than	5 > 3 → True
<	Less than	5 < 10 → True
>=	Greater than or equal	5 >= 5 → True
<=	Less than or equal	5 <= 10 → True

Sample Code

python

Simple if statement

age = 18

if age >= 18:

 print("You are eligible to vote")

 print("Please register to vote")

if-else statement

temperature = 30

if temperature > 25:

 print("It's hot outside!")

 print("Wear light clothes")

else:

 print("It's cool outside!")

 print("Bring a jacket")

Checking multiple conditions

score = 85

```
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
else:
    print("Grade: F")

print("Thank you for taking the test")
```

Output

```
text
You are eligible to vote
Please register to vote
It's hot outside!
Wear light clothes
Grade: B
Thank you for taking the test
```

Explanation

- if checks a condition
- elif (else if) checks additional conditions
- else runs if no conditions are True
- Indentation is crucial (4 spaces standard)
- Conditions are evaluated from top to bottom

TOPIC 19: IF-ELIF-ELSE WITH LOGICAL OPERATORS

Clear Explanation

You can combine multiple conditions using logical operators: and, or, not. This allows for more complex decision-making.

Real-Time Example

A restaurant reservation system:

- If party size > 6 AND has reservation → special table
- If party size > 6 OR has reservation → regular table
- If not (no reservation AND large group) → any table

Logical Operators:

Operator	Description	Example
and	Both conditions must be True	age > 18 and has_license
or	At least one condition True	weekend or holiday
not	Reverses condition	not is_raining

Sample Code

```
python
```

```
# Using AND
```

```
age = 25
```

```
has_license = True
```

```
if age >= 18 and has_license:
```

```
    print("You can drive")
```

```
else:
```

```
    print("You cannot drive")
```

```
# Using OR
```

```
is_weekend = True
```

```
is_holiday = False
```

```
if is_weekend or is_holiday:
```

```
    print("No work today!")
```

else:

```
    print("Time to work")
```

Using NOT

```
is_raining = False
```

if not is_raining:

```
    print("Let's go for a walk")
```

else:

```
    print("Stay inside")
```

Complex conditions

```
marks = 85
```

```
attendance = 90
```

if marks >= 80 and attendance >= 75:

```
    print("Excellent performance")
```

elif marks >= 80 or attendance >= 75:

```
    print("Good, but could improve")
```

else:

```
    print("Needs improvement")
```

Nested if statements

```
num = 15
```

if num > 0:

```
    print("Positive number")
```

```
    if num % 2 == 0:
```

```
        print("Even number")
```

```
    else:
```

```
        print("Odd number")
```

else:

```
    print("Negative or zero")
```

Output

text

You can drive

No work today!

Let's go for a walk

Excellent performance

Positive number

Odd number

Explanation

- and requires both conditions to be True
- or requires at least one condition to be True
- not flips True to False and vice versa
- Nested ifs can check additional conditions
- Use parentheses to group complex conditions

TOPIC 20: FOR LOOPS

Clear Explanation

A for loop is used to iterate over a sequence (list, tuple, string, etc.) or any iterable object. It repeats a block of code for each item in the sequence.

Real-Time Example

Like going through a checklist:

- For each item on your shopping list → buy that item
- For each student in class → mark attendance

Sample Code

```
python
```

```
# Looping through a list
```

```
fruits = ["apple", "banana", "orange", "mango"]
```

```
print("Fruits list:")
```

```
for fruit in fruits:
```

```
print(f"- {fruit}")
```

Looping through a string

```
word = "Python"
```

```
print("\nCharacters in 'Python':")
```

```
for letter in word:
```

```
    print(letter)
```

Looping through a tuple

```
colors = ("red", "green", "blue")
```

```
print("\nColors:")
```

```
for color in colors:
```

```
    print(color)
```

Looping through a dictionary

```
student = {"name": "Alice", "age": 20, "grade": "A"}
```

```
print("\nStudent info:")
```

```
for key, value in student.items():
```

```
    print(f"{key}: {value}")
```

Using break in loop

```
print("\nBreak when finding banana:")
```

```
fruits = ["apple", "banana", "orange", "mango"]
```

```
for fruit in fruits:
```

```
    if fruit == "banana":
```

```
        print("Found banana!")
```

```
        break
```

```
    print(fruit)
```

Using continue

```
print("\nSkip banana:")
```

```
for fruit in fruits:
```

```
    if fruit == "banana":
```

```
        continue
```

```
    print(fruit)
```

Output

text

Fruits list:

- apple

- banana

- orange

- mango

Characters in 'Python':

P

y

t

h

o

n

Colors:

red

green

blue

Student info:

name: Alice

age: 20

grade: A

Break when finding banana:

apple

Found banana!

Skip banana:

apple

orange

mango

Explanation

- for loops iterate through sequences
- break exits the loop completely
- continue skips to next iteration
- else can be used with loops (executes if no break) **TOPIC 21: FOR LOOPS WITH RANGE()**
- **Clear Explanation**
- The range() function generates a sequence of numbers, commonly used with for loops for counting or repeating an action a specific number of times.
- **Real-Time Example**
- Like counting to 10:
- "I'll count from 1 to 10"
- "I'll do this 5 times"
- **Range() Syntax:**
- range(stop) → 0 to stop-1
- range(start, stop) → start to stop-1
- range(start, stop, step) → start to stop-1, increment by step
- **Sample Code**
- python
- *# Basic range*
- print("Range(5):")
- for i in range(5):
- print(i, end=" ")
- print()
-

- *# Range with start and stop*
- `print("\nRange(2, 7):")`
- `for i in range(2, 7):`
- `print(i, end=" ")`
- `print()`
-
- *# Range with step*
- `print("\nRange(0, 10, 2):")`
- `for i in range(0, 10, 2):`
- `print(i, end=" ")`
- `print()`
-
- *# Counting backwards*
- `print("\nRange(10, 0, -2):")`
- `for i in range(10, 0, -2):`
- `print(i, end=" ")`
- `print()`
-
- *# Practical examples*
- `print("\nMultiplication table for 5:")`
- `for i in range(1, 11):`
- `print(f"5 x {i} = {5 * i}")`
-
- *# Sum of numbers 1 to 100*
- `total = 0`
- `for i in range(1, 101):`
- `total += i`
- `print(f"\nSum of 1 to 100: {total}")`
-
- *# Creating a list with range*
- `even_numbers = list(range(2, 21, 2))`

- `print(f"Even numbers: {even_numbers}")`
-
- *# Nested loops*
- `print("\nMultiplication table (1-3):")`
- `for i in range(1, 4):`
- `for j in range(1, 4):`
- `print(f"{i} x {j} = {i * j}")`
- `print()`
- **Output**
- text
- `Range(5):`
- 0 1 2 3 4
-
- `Range(2, 7):`
- 2 3 4 5 6
-
- `Range(0, 10, 2):`
- 0 2 4 6 8
-
- `Range(10, 0, -2):`
- 10 8 6 4 2
-
- Multiplication table for 5:
- 5 x 1 = 5
- 5 x 2 = 10
- 5 x 3 = 15
- 5 x 4 = 20
- 5 x 5 = 25
- 5 x 6 = 30
- 5 x 7 = 35
- 5 x 8 = 40

- $5 \times 9 = 45$
- $5 \times 10 = 50$
-
- Sum of 1 to 100: 5050
- Even numbers: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
-
- Multiplication table (1-3):
- $1 \times 1 = 1$
- $1 \times 2 = 2$
- $1 \times 3 = 3$
-
- $2 \times 1 = 2$
- $2 \times 2 = 4$
- $2 \times 3 = 6$
-
- $3 \times 1 = 3$
- $3 \times 2 = 6$
- $3 \times 3 = 9$
- **Explanation**
- range() generates numbers on the fly (memory efficient)
- Default start is 0, step is 1
- Use list(range()) to see all values
- Nested loops run inner loop completely for each outer iteration

• TOPIC 22: WHILE LOOPS

• Clear Explanation

- A while loop repeats a block of code as long as a condition is True. It's useful when you don't know how many times you need to loop.

• Real-Time Example

- Like a vending machine:
- While money is inserted → allow selection

- When money runs out → stop
- **Sample Code**
- python
- *# Simple while loop*
- count = 1
- print("Counting to 5:")
- while count <= 5:
- print(count)
- count += 1
-
- *# User input validation*
- print("\n--- User Validation ---")
- password = ""
- while password != "secret":
- password = input("Enter password: ")
- if password != "secret":
- print("Wrong password. Try again.")
- print("Access granted!")
-
- *# Infinite loop with break*
- print("\n--- Number Guessing ---")
- import random
- target = random.randint(1, 10)
- attempts = 0
-
- while True:
- guess = int(input("Guess a number (1-10): "))
- attempts += 1
- if guess == target:
- print(f"Correct! You guessed in {attempts} attempts")
- break

- elif guess < target:
- print("Too low, try again")
- else:
- print("Too high, try again")
-
- *# Sum until negative number*
- print("\n--- Sum Calculator ---")
- total = 0
- num = 0
-
- while num >= 0:
- total += num
- num = int(input("Enter number (negative to stop): "))
- print(f"Total sum: {total}")
-
- *# While with else*
- print("\n--- While with else ---")
- n = 3
- while n > 0:
- print(f"Countdown: {n}")
- n -= 1
- else:
- print("Liftoff!")
- **Output (Example)**
- text
- Counting to 5:
- 1
- 2
- 3
- 4
- 5

-
- --- User Validation ---
- Enter password: hello
- Wrong password. Try again.
- Enter password: secret
- Access granted!
-
- --- Number Guessing ---
- Guess a number (1-10): 5
- Too low, try again
- Guess a number (1-10): 8
- Too high, try again
- Guess a number (1-10): 7
- Correct! You guessed in 3 attempts
-
- --- Sum Calculator ---
- Enter number (negative to stop): 10
- Enter number (negative to stop): 20
- Enter number (negative to stop): -1
- Total sum: 30
-
- --- While with else ---
- Countdown: 3
- Countdown: 2
- Countdown: 1
- Liftoff!
- **Explanation**
- while loops continue while condition is True
- Use break to exit loop early
- else executes when loop ends naturally (not by break)
- Be careful of infinite loops

- Great for input validation and unknown iterations

-

- **TOPIC 23: FUNCTIONS**

- **Clear Explanation**

- Functions are reusable blocks of code that perform a specific task. They help organize code, avoid repetition, and make programs easier to understand and maintain.

- **Real-Time Example**

- Like a recipe for making coffee:
- You define the steps once
- Anyone can use the recipe anytime
- Same steps → same result

- **Sample Code**

- python
- *# Defining and calling a simple function*
- `def greet():`
- `"""This function prints a greeting"""`
- `print("Hello!")`
- `print("Welcome to Python")`
-
- *# Call the function*
- `greet()`
- `greet() # Can call multiple times`
-
- *# Function with parameters*
- `def greet_person(name):`
- `print(f"Hello, {name}!")`
- `print("Nice to meet you")`
-
- `greet_person("Alice")`
- `greet_person("Bob")`
-

- *# Function with multiple parameters*
- `def add_numbers(a, b):`
- `result = a + b`
- `print(f'{a} + {b} = {result}')`
-
- `add_numbers(5, 3)`
- `add_numbers(10, 20)`
-
- *# Function with return value*
- `def multiply(x, y):`
- `return x * y`
-
- `product = multiply(4, 5)`
- `print("Product:", product)`
-
- *# Function with default parameter*
- `def greet_with_title(name, title="Mr."):`
- `print(f'Hello, {title} {name}')`
-
- `greet_with_title("Smith")`
- `greet_with_title("Johnson", "Dr.")`
-
- *# Function with multiple returns*
- `def get_min_max(numbers):`
- `min_num = min(numbers)`
- `max_num = max(numbers)`
- `return min_num, max_num`
-
- `minimum, maximum = get_min_max([3, 1, 7, 4, 9])`
- `print(f'Min: {minimum}, Max: {maximum}')`
- **Output**

- text
- Hello!
- Welcome to Python
- Hello!
- Welcome to Python
- Hello, Alice!
- Nice to meet you
- Hello, Bob!
- Nice to meet you
- $5 + 3 = 8$
- $10 + 20 = 30$
- Product: 20
- Hello, Mr. Smith
- Hello, Dr. Johnson
- Min: 1, Max: 9
- **Explanation**
- def keyword defines a function
- Parameters are inputs to the function
- return sends back a value (optional)
- Functions should do one thing well
- Use docstrings (""" ... """) to document functions

-
- **TOPIC 24: FUNCTION ARGUMENTS**
 - **Clear Explanation**
 - Python provides flexible ways to pass arguments to functions: positional, keyword, default, and variable-length arguments.
 - **Real-Time Example**
 - Like ordering food:
 - Positional: burger, fries, drink (in specific order)
 - Keyword: drink=soda, fries=large (specify by name)
 - Default: if you don't specify size, get regular

- **Sample Code**
- python
- *# Positional arguments (order matters)*
- `def create_profile(name, age, city):`
- `print(f"Name: {name}, Age: {age}, City: {city}")`
-
- `create_profile("John", 25, "New York")`
-
- *# Keyword arguments (order doesn't matter)*
- `create_profile(age=30, city="Boston", name="Sarah")`
-
- *# Mixing positional and keyword*
- `create_profile("Mike", city="Chicago", age=28)`
-
- *# Default arguments*
- `def make_coffee(size="medium", sugar=1):`
- `print(f"Making {size} coffee with {sugar} sugar(s)")`
-
- `make_coffee()`
- `make_coffee("large")`
- `make_coffee("small", 0)`
- `make_coffee(sugar=2, size="large")`
-
- *# Variable-length arguments (*args)*
- `def sum_all(*numbers):`
- `total = 0`
- `for num in numbers:`
- `total += num`
- `return total`
-
- `print("Sum of 1,2,3:", sum_all(1, 2, 3))`

- `print("Sum of 10,20,30,40:", sum_all(10, 20, 30, 40))`
-
- *# Variable-length keyword arguments (**kwargs)*
- `def print_info(**info):`
- `for key, value in info.items():`
- `print(f"{key}: {value}")`
-
- `print_info(name="Alice", age=25, city="London")`
- `print_info(product="Laptop", price=999, brand="Dell")`
-
- *# Combining all types*
- `def flexible_function(a, b=10, *args, **kwargs):`
- `print(f"a: {a}")`
- `print(f"b: {b}")`
- `print(f"args: {args}")`
- `print(f"kwargs: {kwargs}")`
-
- `flexible_function(1, 2, 3, 4, 5, name="John", age=30)`

- **Output**

- text
- Name: John, Age: 25, City: New York
- Name: Sarah, Age: 30, City: Boston
- Name: Mike, Age: 28, City: Chicago
- Making medium coffee with 1 sugar(s)
- Making large coffee with 1 sugar(s)
- Making small coffee with 0 sugar(s)
- Making large coffee with 2 sugar(s)
- Sum of 1,2,3: 6
- Sum of 10,20,30,40: 100
- name: Alice
- age: 25

- city: London
- product: Laptop
- price: 999
- brand: Dell
- a: 1
- b: 2
- args: (3, 4, 5)
- kwargs: {'name': 'John', 'age': 30}
- **Explanation**
- *args collects extra positional arguments as tuple
- **kwargs collects extra keyword arguments as dict
- Default parameters must come after required ones
- Keyword arguments make code more readable

• **TOPIC 25: LAMBDA FUNCTIONS**

• **Clear Explanation**

- Lambda functions are small, anonymous functions defined without a name using the lambda keyword. They can have any number of parameters but only one expression.

• **Real-Time Example**

- Like a quick sticky note:
- Small, one-line instructions
- Use once and discard
- No formal function definition needed

• **Sample Code**

- python
- *# Basic lambda function*
- *# Traditional function*
- def square(x):
- return x * x
-
- *# Lambda equivalent*

- `square_lambda = lambda x: x * x`
-
- `print("Square of 5:", square(5))`
- `print("Square using lambda:", square_lambda(5))`
-
- *# Lambda with multiple parameters*
- `add = lambda a, b: a + b`
- `print("Sum of 10 and 20:", add(10, 20))`
-
- *# Using lambda with built-in functions*
- `numbers = [1, 2, 3, 4, 5]`
-
- *# map() - apply function to all items*
- `squared = list(map(lambda x: x ** 2, numbers))`
- `print("Squared numbers:", squared)`
-
- *# filter() - filter items based on condition*
- `even_numbers = list(filter(lambda x: x % 2 == 0, numbers))`
- `print("Even numbers:", even_numbers)`
-
- *# Sorting with lambda*
- `students = [`
- `{"name": "Alice", "grade": 85},`
- `{"name": "Bob", "grade": 92},`
- `{"name": "Charlie", "grade": 78}`
- `]`
-
- *# Sort by grade*
- `students.sort(key=lambda x: x["grade"])`
- `print("Students sorted by grade:")`
- `for student in students:`

- `print(f" {student['name']}: {student['grade']}")`
-
- *# Sort by name*
- `students.sort(key=lambda x: x["name"])`
- `print("\nStudents sorted by name:")`
- `for student in students:`
- `print(f" {student['name']}: {student['grade']}")`
-
- *# Using lambda with max/min*
- `numbers = [10, 25, 3, 48, 19]`
- `max_num = max(numbers, key=lambda x: x)`
- `print("Maximum:", max_num)`
-
- *# Lambda in dictionary operations*
- `prices = {"apple": 0.50, "banana": 0.30, "orange": 0.40}`
- `cheapest = min(prices, key=lambda x: prices[x])`
- `print("Cheapest fruit:", cheapest)`
- **Output**
- text
- Square of 5: 25
- Square using lambda: 25
- Sum of 10 and 20: 30
- Squared numbers: [1, 4, 9, 16, 25]
- Even numbers: [2, 4]
- Students sorted by grade:
- Charlie: 78
- Alice: 85
- Bob: 92
-
- Students sorted by name:
- Alice: 85

- Bob: 92
- Charlie: 78
- Maximum: 48
- Cheapest fruit: banana
- **When to Use Lambda**
- For short, simple operations
- With map(), filter(), sorted()
- When function is used only once
- To create small inline functions
- **Explanation**
- Lambda syntax: lambda arguments: expression
- No return statement (implied)
- Can be used anywhere a function is expected
- Limited to single expression (no statements)

-
- **TOPIC 26: MODULES**
 - **Clear Explanation**
 - Modules are Python files containing functions, classes, and variables. They help organize code into reusable files. Python has a large standard library of modules.
 - **Real-Time Example**
 - Like a toolbox:
 - Each module is a specific tool
 - You import what you need
 - Don't carry all tools at once
 - **Common Built-in Modules:**
 - math - Mathematical functions
 - random - Random number generation
 - datetime - Date and time operations
 - os - Operating system interface
 - sys - System-specific parameters
 - **Sample Code**

- python
- *# Importing entire module*
- import math
-
- print("PI:", math.pi)
- print("Square root of 16:", math.sqrt(16))
- print("Ceil of 4.3:", math.ceil(4.3))
- print("Floor of 4.7:", math.floor(4.7))
-
- *# Importing specific functions*
- from math import sin, cos, radians
-
- angle = radians(45)
- print(f"sin(45°): {sin(angle)}")
- print(f"cos(45°): {cos(angle)}")
-
- *# Import with alias*
- import random as rnd
-
- print("Random number (1-10):", rnd.randint(1, 10))
- print("Random choice:", rnd.choice(["apple", "banana", "orange"]))
- print("Random sample:", rnd.sample(range(100), 5))
-
- *# Working with datetime*
- from datetime import datetime, timedelta
-
- now = datetime.now()
- print("Current date/time:", now)
- print("Year:", now.year)
- print("Month:", now.month)
- print("Day:", now.day)

-
- *# Date arithmetic*
- tomorrow = now + timedelta(days=1)
- print("Tomorrow:", tomorrow)
-
- *# OS module*
- import os
-
- print("Current directory:", os.getcwd())
- *# os.listdir() shows files in directory*
- *# print("Files:", os.listdir("."))*
-
- *# Import all (not recommended)*
- from math import *
- print("Square root of 25:", sqrt(25))
- **Output**
- text
- PI: 3.141592653589793
- Square root of 16: 4.0
- Ceil of 4.3: 5
- Floor of 4.7: 4
- sin(45°): 0.7071067811865476
- cos(45°): 0.7071067811865476
- Random number (1-10): 7
- Random choice: orange
- Random sample: [42, 17, 83, 5, 91]
- Current date/time: 2024-01-15 14:30:25.123456
- Year: 2024
- Month: 1
- Day: 15
- Tomorrow: 2024-01-16 14:30:25.123456

- Current directory: /home/user/projects
- Square root of 25: 5.0
- **Import Styles**
- python
- import module *# Use: module.function()*
- from module import f *# Use: function()*
- from module import * *# Use: function() (not recommended)*
- import module as m *# Use: m.function()*

- **Explanation**

- Modules help organize code
- Standard library has hundreds of modules
- Create your own modules with .py files
- Use pip to install third-party modules

- **TOPIC 27: FILE HANDLING - READING**

- **Clear Explanation**

- File handling allows Python programs to read data from files. You can open files in different modes: read (r), write (w), append (a), etc.

- **Real-Time Example**

- Like reading a book:
- Open the book (open file)
- Read pages (read content)
- Close the book (close file)

- **File Modes:**

- | Mode | Description |
|------|------------------------|
| • r | • Read (default) |
| • w | • Write (overwrites) |
| • a | • Append (adds to end) |

- | | |
|--------|----------------------------|
| • Mode | • Description |
| • x | • Create (fails if exists) |
| • b | • Binary mode |
| • t | • Text mode (default) |

- **Sample Code**

- python
- *# Creating a sample file first*
- `sample_content = """Line 1: Hello World`
- `Line 2: Python is great`
- `Line 3: File handling is easy`
- `Line 4: Learning is fun`
- `Line 5: Keep practicing!"""`
-
- `with open("sample.txt", "w") as file:`
- `file.write(sample_content)`
-
- `print("Sample file created!")`
-
- *# Reading entire file*
- `print("\n--- Reading entire file ---")`
- `with open("sample.txt", "r") as file:`
- `content = file.read()`
- `print(content)`
-
- *# Reading line by line*
- `print("\n--- Reading line by line ---")`
- `with open("sample.txt", "r") as file:`
- `for line in file:`

- `print(f"Line: {line.strip()}")`
-
- *# Reading specific number of characters*
- `print("\n--- Reading 10 characters ---")`
- `with open("sample.txt", "r") as file:`
- `chunk = file.read(10)`
- `print(chunk)`
-
- *# Using readlines() method*
- `print("\n--- Using readlines() ---")`
- `with open("sample.txt", "r") as file:`
- `lines = file.readlines()`
- `print(f"Number of lines: {len(lines)}")`
- `print(f"First line: {lines[0].strip()}")`
- `print(f"Last line: {lines[-1].strip()}")`
-
- *# Checking if file exists*
- `import os`
-
- `if os.path.exists("sample.txt"):`
- `print("\nFile exists!")`
- `else:`
- `print("\nFile does not exist")`
- **Output**
- text
- Sample file created!
-
- --- Reading entire file ---
- Line 1: Hello World
- Line 2: Python is great
- Line 3: File handling is easy

- Line 4: Learning is fun
 - Line 5: Keep practicing!
 -
 - --- Reading line by line ---
 - Line: Line 1: Hello World
 - Line: Line 2: Python is great
 - Line: Line 3: File handling is easy
 - Line: Line 4: Learning is fun
 - Line: Line 5: Keep practicing!
 -
 - --- Reading 10 characters ---
 - Line 1: Hel
 -
 - --- Using readlines() ---
 - Number of lines: 5
 - First line: Line 1: Hello World
 - Last line: Line 5: Keep practicing!
 -
 - File exists!
 - **Best Practices**
 - Always use with statement (auto-closes files)
 - Handle file not found errors
 - Use appropriate reading method
 - Close files when done (if not using with)
 -
-
- **TOPIC 28: FILE HANDLING - WRITING**
 - **Clear Explanation**
 - Python allows you to create, write to, and modify files. Different modes control whether you overwrite, append, or create new files.
 - **Real-Time Example**
 - Like writing in a notebook:

- w mode: New notebook, write from start
- a mode: Add to existing notebook
- x mode: Only write if notebook doesn't exist

- **Sample Code**

- python
- *# Writing to a new file (w mode)*
- `print("--- Writing to file ---")`
- `with open("notes.txt", "w") as file:`
- `file.write("First line\n")`
- `file.write("Second line\n")`
- `file.write("Third line\n")`
-
- *# Verify*
- `with open("notes.txt", "r") as file:`
- `print("After writing:")`
- `print(file.read())`
-
- *# Appending to file (a mode)*
- `print("\n--- Appending to file ---")`
- `with open("notes.txt", "a") as file:`
- `file.write("Fourth line (appended)\n")`
- `file.write("Fifth line (appended)\n")`
-
- *# Verify*
- `with open("notes.txt", "r") as file:`
- `print("After appending:")`
- `print(file.read())`
-
- *# Writing multiple lines at once*
- `print("\n--- Writing multiple lines ---")`
- `lines = [`

- "Line 1: Apple\n",
- "Line 2: Banana\n",
- "Line 3: Orange\n",
- "Line 4: Mango\n"
-]
-
- with open("fruits.txt", "w") as file:
- file.writelines(lines)
-
- *# Verify*
- with open("fruits.txt", "r") as file:
- print("Fruits file:")
- print(file.read())
-
- *# Creating file with x mode (fails if exists)*
- print("\n--- Creating with x mode ---")
- try:
- with open("new_file.txt", "x") as file:
- file.write("This is a new file")
- print("File created successfully")
- except FileExistsError:
- print("File already exists")
-
- *# Writing user input to file*
- print("\n--- Writing user input ---")
- with open("user_input.txt", "w") as file:
- print("Enter text (type 'STOP' to finish):")
- while True:
- line = input("> ")
- if line == "STOP":
- break

- `file.write(line + "\n")`
- **Output**
- text
- --- Writing to file ---
- After writing:
- First line
- Second line
- Third line
-
- --- Appending to file ---
- After appending:
- First line
- Second line
- Third line
- Fourth line (appended)
- Fifth line (appended)
-
- --- Writing multiple lines ---
- Fruits file:
- Line 1: Apple
- Line 2: Banana
- Line 3: Orange
- Line 4: Mango
- --- Creating with x mode ---
- File created successfully
- --- Writing user input ---
- Enter text (type 'STOP' to finish):
- > Hello
- > Python
- > STOP
- **File Modes Summary**

- | | | | |
|--------|--------------------|----------------|---------------------------|
| • Mode | • Description | • File exists? | • Action |
| • r | • Read | • Yes | • Read from start |
| • w | • Write | • Yes/No | • Overwrite/Create |
| • a | • Append | • Yes/No | • Add to end/Create |
| • x | • Exclusive create | • No | • Create, fails if exists |
| • r+ | • Read/Write | • Yes | • Read and write |
| • w+ | • Write/Read | • Yes/No | • Overwrite/Create |
- **Explanation**
 - Always close files (using with handles this)
 - write() writes a string
 - writelines() writes a list of strings
 - Newline \n must be added manually
 - Different modes serve different purposes

TOPIC 31: INHERITANCE

Clear Explanation

Inheritance allows a class to inherit attributes and methods from another class. The parent class (superclass) provides common functionality, and child classes (subclasses) can add or modify specific behavior.

Real-Time Example

Like family traits:

- Parent class: Vehicle (wheels, engine, drive)
- Child class: Car (inherits + trunk space)
- Child class: Motorcycle (inherits + no roof)

Sample Code

python

Parent class

class Animal:

def __init__(self, name):

self.name = name

def eat(self):

return f"{self.name} is eating"

def sleep(self):

return f"{self.name} is sleeping"

def make_sound(self):

return "Some generic sound"

Child class inheriting from Animal

class Dog(Animal):

def __init__(self, name, breed):

super().__init__(name) *# Call parent constructor*

self.breed = breed

def make_sound(self):

return f"{self.name} barks: Woof!"

def fetch(self):

return f"{self.name} is fetching the ball"

Another child class

class Cat(Animal):

def __init__(self, name, color):

super().__init__(name)

```
self.color = color
```

```
def make_sound(self):
```

```
    return f"{self.name} meows: Meow!"
```

```
def scratch(self):
```

```
    return f"{self.name} is scratching the furniture"
```

```
# Creating objects
```

```
dog = Dog("Buddy", "Golden Retriever")
```

```
cat = Cat("Whiskers", "Orange")
```

```
# Using inherited methods
```

```
print(dog.eat())
```

```
print(dog.sleep())
```

```
print(cat.eat())
```

```
# Using child-specific methods
```

```
print(dog.make_sound())
```

```
print(dog.fetch())
```

```
print(cat.make_sound())
```

```
print(cat.scratch())
```

```
# Checking inheritance
```

```
print("\n--- Type checking ---")
```

```
print(isinstance(dog, Dog))    # True
```

```
print(isinstance(dog, Animal)) # True
```

```
print(isinstance(cat, Animal)) # True
```

```
print(isinstance(dog, Cat))    # False
```

```
# Multiple levels of inheritance
```

```

class Vehicle:

    def __init__(self, brand):

        self.brand = brand


    def start(self):

        return "Engine started"


class Car(Vehicle):

    def __init__(self, brand, model):

        super().__init__(brand)

        self.model = model


    def honk(self):

        return "Beep beep!"


class ElectricCar(Car):

    def __init__(self, brand, model, battery_capacity):

        super().__init__(brand, model)

        self.battery = battery_capacity


    def start(self):

        return "Electric motor started silently"


    def charge(self):

        return f"Charging {self.battery} kWh battery"


# Using multi-level inheritance

tesla = ElectricCar("Tesla", "Model 3", 75)

print(tesla.start())    # Overridden method

print(tesla.honk())     # From Car

print(tesla.charge())   # From ElectricCar

```

Output

text

Buddy is eating

Buddy is sleeping

Whiskers is eating

Buddy barks: Woof!

Buddy is fetching the ball

Whiskers meows: Meow!

Whiskers is scratching the furniture

--- Type checking ---

True

True

True

False

Electric motor started silently

Beep beep!

Charging 75 kWh battery

Key Inheritance Concepts

- `super()` calls parent class methods
- Child classes can override parent methods
- Multiple inheritance is possible
- `isinstance()` checks object type
- `issubclass()` checks class relationships

TOPIC 32: LIST COMPREHENSIONS

Clear Explanation

List comprehensions provide a concise way to create lists based on existing sequences. They are more readable and often faster than traditional loops.

Real-Time Example

Like a factory assembly line:

- Input: Raw materials (original list)
- Process: Apply transformation
- Output: Finished products (new list)

Sample Code

python

Basic list comprehension

Traditional way

```
squares = []
```

```
for x in range(10):
```

```
    squares.append(x ** 2)
```

```
print("Traditional squares:", squares)
```

List comprehension

```
squares_comp = [x ** 2 for x in range(10)]
```

```
print("List comprehension squares:", squares_comp)
```

With condition

Get even numbers

```
even_numbers = [x for x in range(20) if x % 2 == 0]
```

```
print("Even numbers:", even_numbers)
```

With if-else

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
result = ["even" if x % 2 == 0 else "odd" for x in numbers]
```

```
print("Even/odd classification:", result)
```

Transforming data

```
names = ["alice", "bob", "charlie", "david"]
```

```
capitalized = [name.capitalize() for name in names]
```

```
print("Capitalized names:", capitalized)
```

With multiple loops

```
pairs = [(x, y) for x in [1, 2, 3] for y in [4, 5, 6]]  
print("All pairs:", pairs)
```

Flattening a list

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]  
flattened = [num for row in matrix for num in row]  
print("Flattened matrix:", flattened)
```

With complex conditions

```
words = ["apple", "banana", "cherry", "date", "elderberry"]  
long_words = [word.upper() for word in words if len(word) > 5]  
print("Long words (uppercase):", long_words)
```

Nested list comprehensions

```
matrix2 = [[i * j for j in range(1, 4)] for i in range(1, 4)]  
print("Multiplication table matrix:")  
  
for row in matrix2:  
    print(row)
```

Practical example - filtering and transforming

```
scores = [85, 92, 78, 65, 95, 88, 70]  
grades = ['A' if s >= 90 else 'B' if s >= 80 else 'C' if s >= 70 else 'F' for s in scores]  
print("Scores:", scores)  
print("Grades:", grades)
```

Set comprehension

```
unique_lengths = {len(word) for word in words}  
print("Unique word lengths:", unique_lengths)
```

Dictionary comprehension

```
square_dict = {x: x**2 for x in range(1, 6)}  
print("Number squares dictionary:", square_dict)
```

Output

text

Traditional squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

List comprehension squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

Even numbers: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

Even/odd classification: ['odd', 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even']

Capitalized names: ['Alice', 'Bob', 'Charlie', 'David']

All pairs: [(1, 4), (1, 5), (1, 6), (2, 4), (2, 5), (2, 6), (3, 4), (3, 5), (3, 6)]

Flattened matrix: [1, 2, 3, 4, 5, 6, 7, 8, 9]

Long words (uppercase): ['BANANA', 'CHERRY', 'ELDERBERRY']

Multiplication table matrix:

[1, 2, 3]

[2, 4, 6]

[3, 6, 9]

Scores: [85, 92, 78, 65, 95, 88, 70]

Grades: ['B', 'A', 'C', 'F', 'A', 'B', 'C']

Unique word lengths: {5, 6, 7, 4}

Number squares dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Syntax

text

[expression for item in iterable if condition]

Benefits

- More concise and readable
- Often faster than loops
- Great for data transformation
- Can include conditions

TOPIC 33: DECORATORS

Clear Explanation

Decorators are functions that modify the behavior of other functions. They allow you to add functionality to existing functions without changing their code.

Real-Time Example

Like adding a gift wrapper:

- Original: A gift (function)
- Decorator: Wrapping paper (adds functionality)
- Result: Wrapped gift (enhanced function)

Sample Code

python

Basic decorator

```
def my_decorator(func):  
    def wrapper():  
        print("Something is happening before the function")  
        func()  
        print("Something is happening after the function")  
    return wrapper
```

@my_decorator

```
def say_hello():  
    print("Hello!")
```

```
print("--- Basic decorator ---")
```

```
say_hello()
```

Decorator with arguments

```
def timer(func):  
    import time  
    def wrapper(*args, **kwargs):  
        start = time.time()  
        result = func(*args, **kwargs)  
        end = time.time()
```

```
    print(f"{func.__name__} took {end - start:.4f} seconds")
    return result
return wrapper
```

```
@timer
```

```
def slow_function(seconds):
```

```
    import time
    time.sleep(seconds)
    return "Done!"
```

```
print("\n--- Timer decorator ---")
```

```
result = slow_function(1)
```

```
print(f"Result: {result}")
```

```
# Decorator with parameters
```

```
def repeat(n):
```

```
    def decorator(func):
```

```
        def wrapper(*args, **kwargs):
            results = []
            for _ in range(n):
                results.append(func(*args, **kwargs))
            return results
```

```
        return wrapper
```

```
    return decorator
```

```
@repeat(3)
```

```
def greet(name):
```

```
    return f"Hello, {name}!"
```

```
print("\n--- Repeat decorator ---")
```

```
greetings = greet("Alice")
```

```
print(greetings)
```

```
# Multiple decorators
```

```
def uppercase(func):
```

```
    def wrapper():
```

```
        result = func()
```

```
        return result.upper()
```

```
    return wrapper
```

```
def exclamation(func):
```

```
    def wrapper():
```

```
        result = func()
```

```
        return result + "!!!"
```

```
    return wrapper
```

```
@uppercase
```

```
@exclamation
```

```
def get_message():
```

```
    return "hello world"
```

```
print("\n--- Multiple decorators ---")
```

```
print(get_message()) # Executes exclamation first, then uppercase
```

```
# Practical decorator - authentication
```

```
def requires_auth(func):
```

```
    def wrapper(user, *args, **kwargs):
```

```
        if user.get("authenticated", False):
```

```
            return func(user, *args, **kwargs)
```

```
        else:
```

```
            return "Access denied. Please log in."
```

```
    return wrapper
```

```

@requires_auth
def view_profile(user):
    return f"Profile: {user['name']}, Email: {user['email']}"

print("\n--- Authentication decorator ---")

user1 = {"name": "Alice", "email": "alice@example.com", "authenticated": True}
user2 = {"name": "Bob", "email": "bob@example.com", "authenticated": False}

print(view_profile(user1))
print(view_profile(user2))

```

Decorator with class

```

class CountCalls:
    def __init__(self, func):
        self.func = func
        self.count = 0

    def __call__(self, *args, **kwargs):
        self.count += 1
        print(f"Function {self.func.__name__} called {self.count} times")
        return self.func(*args, **kwargs)

```

@CountCalls

```

def test_function():
    return "Function executed"

```

```

print("\n--- Class decorator ---")

test_function()
test_function()
test_function()

```

Output

text

--- Basic decorator ---

Something is happening before the function

Hello!

Something is happening after the function

--- Timer decorator ---

slow_function took 1.0012 seconds

Result: Done!

--- Repeat decorator ---

['Hello, Alice!', 'Hello, Alice!', 'Hello, Alice!']

--- Multiple decorators ---

HELLO WORLD!!!

--- Authentication decorator ---

Profile: Alice, Email: alice@example.com

Access denied. Please log in.

--- Class decorator ---

Function test_function called 1 times

Function test_function called 2 times

Function test_function called 3 times

Common Use Cases

- Logging
- Timing functions
- Access control
- Caching
- Input validation

TOPIC 34: GENERATORS

Clear Explanation

Generators are functions that produce a sequence of results lazily (one at a time) instead of computing all at once. They use yield instead of return.

Real-Time Example

Like a streaming service vs downloading:

- Generator: Streams movie (one scene at a time)
- List: Downloads entire movie (all at once)
- More memory efficient for large data

Sample Code

```
python
```

```
# Basic generator
```

```
def count_up_to(n):
```

```
    """Generate numbers from 1 to n"""
```

```
    i = 1
```

```
    while i <= n:
```

```
        yield i
```

```
        i += 1
```

```
print("--- Basic generator ---")
```

```
counter = count_up_to(5)
```

```
print(next(counter)) # 1
```

```
print(next(counter)) # 2
```

```
print(next(counter)) # 3
```

```
# Using generator in loop
```

```
print("\n--- Using in loop ---")
```

```
for num in count_up_to(5):
```

```
    print(num, end=" ")
```

```
print()
```

Generator vs list memory comparison

```
import sys
```

```
def fibonacci_list(n):
```

```
    """Create list of Fibonacci numbers"""
```

```
    fib = [0, 1]
```

```
    for i in range(2, n):
```

```
        fib.append(fib[-1] + fib[-2])
```

```
    return fib[:n]
```

```
def fibonacci_gen(n):
```

```
    """Generate Fibonacci numbers"""
```

```
    a, b = 0, 1
```

```
    for _ in range(n):
```

```
        yield a
```

```
        a, b = b, a + b
```

Memory usage

```
list_fib = fibonacci_list(100000)
```

```
gen_fib = fibonacci_gen(100000)
```

```
print("\n--- Memory comparison ---")
```

```
print(f"List size: {sys.getsizeof(list_fib)} bytes")
```

```
print(f"Generator size: {sys.getsizeof(gen_fib)} bytes")
```

Infinite generator

```
def infinite_numbers():
```

```
    """Generate infinite sequence of numbers"""
```

```
    n = 0
```

```
    while True:
```

```
yield n
```

```
n += 1
```

```
print("\n--- Infinite generator (first 10) ---")
```

```
numbers = infinite_numbers()
```

```
for _ in range(10):
```

```
    print(next(numbers), end=" ")
```

```
print("...")
```

```
# Generator expression (like list comprehension)
```

```
print("\n--- Generator expression ---")
```

```
squares_gen = (x**2 for x in range(10))
```

```
print("Type:", type(squares_gen))
```

```
print("Squares:", list(squares_gen))
```

```
# Practical example - reading large files
```

```
def read_large_file(file_path):
```

```
    """Read file line by line without loading entire file"""
```

```
    with open(file_path, 'r') as file:
```

```
        for line in file:
```

```
            yield line.strip()
```

```
# Creating a sample file
```

```
with open("large_sample.txt", "w") as f:
```

```
    for i in range(100):
```

```
        f.write(f"This is line number {i}\n")
```

```
print("\n--- Reading large file (first 5 lines) ---")
```

```
for i, line in enumerate(read_large_file("large_sample.txt")):
```

```
    if i >= 5:
```

```
        break
```

```
print(line)
```

```
# Generator pipeline
```

```
def even_numbers(numbers):
```

```
    for n in numbers:
```

```
        if n % 2 == 0:
```

```
            yield n
```

```
def square(numbers):
```

```
    for n in numbers:
```

```
        yield n ** 2
```

```
def cube(numbers):
```

```
    for n in numbers:
```

```
        yield n ** 3
```

```
print("\n--- Generator pipeline ---")
```

```
numbers = range(10)
```

```
even = even_numbers(numbers)
```

```
squared = square(even)
```

```
cubed = cube(even_numbers(numbers))
```

```
print("Even numbers:", list(even))
```

```
print("Squares of evens:", list(squared))
```

```
print("Cubes of evens:", list(cubed))
```

Output

```
text
```

```
--- Basic generator ---
```

```
1
```

```
2
```

```
3
```

--- Using in loop ---

1 2 3 4 5

--- Memory comparison ---

List size: 800056 bytes

Generator size: 120 bytes

--- Infinite generator (first 10) ---

0 1 2 3 4 5 6 7 8 9 ...

--- Generator expression ---

Type: <class 'generator'>

Squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

--- Reading large file (first 5 lines) ---

This is line number 0

This is line number 1

This is line number 2

This is line number 3

This is line number 4

--- Generator pipeline ---

Even numbers: [0, 2, 4, 6, 8]

Squares of evens: [0, 4, 16, 36, 64]

Cubes of evens: [0, 8, 64, 216, 512]

Key Points

- yield returns value and pauses function
- Generator resumes where it left off
- Much more memory efficient for large datasets
- Can represent infinite sequences

- Can be used in pipelines
-

TOPIC 35: WORKING WITH DATES AND TIME

Clear Explanation

Python's datetime module provides classes for working with dates, times, and time intervals. It handles date arithmetic, formatting, and parsing.

Real-Time Example

Like a digital calendar:

- Track dates and events
- Calculate days until an event
- Format dates differently for different countries

Sample Code

```
python
from datetime import datetime, date, time, timedelta
import time as timer
```

```
# Current date and time
```

```
print("--- Current date and time ---")
now = datetime.now()
print("Current datetime:", now)
print("Current date:", now.date())
print("Current time:", now.time())
print("Year:", now.year)
print("Month:", now.month)
print("Day:", now.day)
print("Hour:", now.hour)
print("Minute:", now.minute)
print("Second:", now.second)
```

```
# Creating specific dates
```

```
print("\n--- Creating dates ---")
```

```
birthday = date(1995, 5, 15)
```

```
print("Birthday:", birthday)
```

```
meeting = datetime(2024, 12, 25, 14, 30, 0)
```

```
print("Meeting:", meeting)
```

```
# Date arithmetic
```

```
print("\n--- Date arithmetic ---")
```

```
today = date.today()
```

```
future = today + timedelta(days=30)
```

```
past = today - timedelta(days=7)
```

```
print("Today:", today)
```

```
print("30 days from now:", future)
```

```
print("7 days ago:", past)
```

```
# Time difference
```

```
birth = date(1990, 1, 1)
```

```
age = today - birth
```

```
print(f"Age in days: {age.days}")
```

```
print(f"Age in years: {age.days // 365}")
```

```
# Formatting dates
```

```
print("\n--- Formatting dates ---")
```

```
now = datetime.now()
```

```
print("Default:", now)
```

```
print("YYYY-MM-DD:", now.strftime("%Y-%m-%d"))
```

```
print("MM/DD/YYYY:", now.strftime("%m/%d/%Y"))
```

```
print("Month name:", now.strftime("%B %d, %Y"))
```

```
print("Weekday:", now.strftime("%A"))
```

```
print("12-hour time:", now.strftime("%I:%M %p"))
```

```
print("Custom:", now.strftime("%A, %B %d, %Y at %I:%M %p"))
```

```
# Parsing strings to dates
```

```
print("\n--- Parsing dates ---")
```

```
date_string = "2024-12-25"
```

```
parsed_date = datetime.strptime(date_string, "%Y-%m-%d")
```

```
print(f"Parsed: {parsed_date}")
```

```
date_string2 = "Dec 25, 2024"
```

```
parsed_date2 = datetime.strptime(date_string2, "%b %d, %Y")
```

```
print(f"Parsed: {parsed_date2}")
```

```
# Working with time only
```

```
print("\n--- Time objects ---")
```

```
current_time = datetime.now().time()
```

```
print("Current time:", current_time)
```

```
alarm = time(7, 30, 0)
```

```
print("Alarm set for:", alarm)
```

```
# Comparing dates
```

```
print("\n--- Comparing dates ---")
```

```
date1 = date(2024, 1, 1)
```

```
date2 = date(2024, 12, 31)
```

```
print(f"{date1} < {date2}: {date1 < date2}")
```

```
print(f"{date1} > {date2}: {date1 > date2}")
```

```
print(f"{date1} == {date2}: {date1 == date2}")
```

```
# Timezone aware dates
```

```
print("\n--- Timezone (UTC) ---")
```

```
from datetime import timezone
```

```
utc_now = datetime.now(timezone.utc)
```

```
print("UTC now:", utc_now)
```

```
# Practical example - countdown
```

```
def days_until_event(event_date):
```

```
    today = date.today()
```

```
    if event_date < today:
```

```
        return f"Event was { (today - event_date).days } days ago"
```

```
    else:
```

```
        return f"Event in { (event_date - today).days } days"
```

```
print("\n--- Countdown example ---")
```

```
new_year = date(2025, 1, 1)
```

```
print(f"New Year 2025: {days_until_event(new_year)}")
```

```
# Timer example
```

```
print("\n--- Timer example ---")
```

```
print("Starting 3 second countdown...")
```

```
for i in range(3, 0, -1):
```

```
    print(i)
```

```
    timer.sleep(1)
```

```
print("Time's up!")
```

Output

text

--- Current date and time ---

Current datetime: 2024-01-15 14:30:25.123456

Current date: 2024-01-15

Current time: 14:30:25.123456

Year: 2024

Month: 1

Day: 15

Hour: 14

Minute: 30

Second: 25

--- Creating dates ---

Birthday: 1995-05-15

Meeting: 2024-12-25 14:30:00

--- Date arithmetic ---

Today: 2024-01-15

30 days from now: 2024-02-14

7 days ago: 2024-01-08

Age in days: 12428

Age in years: 34

--- Formatting dates ---

Default: 2024-01-15 14:30:25.123456

YYYY-MM-DD: 2024-01-15

MM/DD/YYYY: 01/15/2024

Month name: January 15, 2024

Weekday: Monday

12-hour time: 02:30 PM

Custom: Monday, January 15, 2024 at 02:30 PM

--- Parsing dates ---

Parsed: 2024-12-25 00:00:00

Parsed: 2024-12-25 00:00:00

--- Time objects ---

Current time: 14:30:25.123456

Alarm set for: 07:30:00

--- Comparing dates ---

2024-01-01 < 2024-12-31: True

2024-01-01 > 2024-12-31: False

2024-01-01 == 2024-12-31: False

--- Timezone (UTC) ---

UTC now: 2024-01-15 14:30:25.123456+00:00

--- Countdown example ---

New Year 2025: Event in 351 days

--- Timer example ---

Starting 3 second countdown...

3

2

1

Time's up!

Common Format Codes

Code	Meaning	Example
%Y	Year (4-digit)	2024
%y	Year (2-digit)	24
%m	Month (01-12)	01
%d	Day (01-31)	15
%H	Hour (00-23)	14

Code	Meaning	Example
%I	Hour (01-12)	02
%M	Minute (00-59)	30
%S	Second (00-59)	25
%p	AM/PM	PM
%A	Full weekday	Monday
%a	Short weekday	Mon
%B	Full month	January
%b	Short month	Jan

TOPIC 31: INHERITANCE

Clear Explanation

Inheritance allows a class to inherit attributes and methods from another class. The parent class (superclass) provides common functionality, and child classes (subclasses) can add or modify specific behavior.

Real-Time Example

Like family traits:

- Parent class: Vehicle (wheels, engine, drive)
- Child class: Car (inherits + trunk space)
- Child class: Motorcycle (inherits + no roof)

Sample Code

python

Parent class

class Animal:

def __init__(self, name):

self.name = name

```
def eat(self):  
    return f"{self.name} is eating"
```

```
def sleep(self):  
    return f"{self.name} is sleeping"
```

```
def make_sound(self):  
    return "Some generic sound"
```

Child class inheriting from Animal

```
class Dog(Animal):  
    def __init__(self, name, breed):  
        super().__init__(name) # Call parent constructor  
        self.breed = breed  
  
    def make_sound(self):  
        return f"{self.name} barks: Woof!"  
  
    def fetch(self):  
        return f"{self.name} is fetching the ball"
```

Another child class

```
class Cat(Animal):  
    def __init__(self, name, color):  
        super().__init__(name)  
        self.color = color  
  
    def make_sound(self):  
        return f"{self.name} meows: Meow!"
```

```
def scratch(self):  
    return f"{self.name} is scratching the furniture"
```

Creating objects

```
dog = Dog("Buddy", "Golden Retriever")  
cat = Cat("Whiskers", "Orange")
```

Using inherited methods

```
print(dog.eat())  
print(dog.sleep())  
print(cat.eat())
```

Using child-specific methods

```
print(dog.make_sound())  
print(dog.fetch())  
print(cat.make_sound())  
print(cat.scratch())
```

Checking inheritance

```
print("\n--- Type checking ---")  
print(isinstance(dog, Dog))    # True  
print(isinstance(dog, Animal)) # True  
print(isinstance(cat, Animal)) # True  
print(isinstance(dog, Cat))    # False
```

Multiple levels of inheritance

```
class Vehicle:  
    def __init__(self, brand):  
        self.brand = brand  
  
    def start(self):
```

```
    return "Engine started"
```

```
class Car(Vehicle):
```

```
    def __init__(self, brand, model):
```

```
        super().__init__(brand)
```

```
        self.model = model
```

```
    def honk(self):
```

```
        return "Beep beep!"
```

```
class ElectricCar(Car):
```

```
    def __init__(self, brand, model, battery_capacity):
```

```
        super().__init__(brand, model)
```

```
        self.battery = battery_capacity
```

```
    def start(self):
```

```
        return "Electric motor started silently"
```

```
    def charge(self):
```

```
        return f"Charging {self.battery} kWh battery"
```

```
# Using multi-level inheritance
```

```
tesla = ElectricCar("Tesla", "Model 3", 75)
```

```
print(tesla.start())    # Overridden method
```

```
print(tesla.honk())     # From Car
```

```
print(tesla.charge())   # From ElectricCar
```

Output

text

Buddy is eating

Buddy is sleeping

Whiskers is eating

Buddy barks: Woof!

Buddy is fetching the ball

Whiskers meows: Meow!

Whiskers is scratching the furniture

--- Type checking ---

True

True

True

False

Electric motor started silently

Beep beep!

Charging 75 kWh battery

Key Inheritance Concepts

- `super()` calls parent class methods
- Child classes can override parent methods
- Multiple inheritance is possible
- `isinstance()` checks object type
- `issubclass()` checks class relationships

TOPIC 32: LIST COMPREHENSIONS

Clear Explanation

List comprehensions provide a concise way to create lists based on existing sequences. They are more readable and often faster than traditional loops.

Real-Time Example

Like a factory assembly line:

- Input: Raw materials (original list)
- Process: Apply transformation
- Output: Finished products (new list)

Sample Code

python

Basic list comprehension

Traditional way

```
squares = []  
for x in range(10):  
    squares.append(x ** 2)  
print("Traditional squares:", squares)
```

List comprehension

```
squares_comp = [x ** 2 for x in range(10)]  
print("List comprehension squares:", squares_comp)
```

With condition

Get even numbers

```
even_numbers = [x for x in range(20) if x % 2 == 0]  
print("Even numbers:", even_numbers)
```

With if-else

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
result = ["even" if x % 2 == 0 else "odd" for x in numbers]  
print("Even/odd classification:", result)
```

Transforming data

```
names = ["alice", "bob", "charlie", "david"]  
capitalized = [name.capitalize() for name in names]  
print("Capitalized names:", capitalized)
```

With multiple loops

```
pairs = [(x, y) for x in [1, 2, 3] for y in [4, 5, 6]]  
print("All pairs:", pairs)
```

Flattening a list

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = [num for row in matrix for num in row]
print("Flattened matrix:", flattened)
```

With complex conditions

```
words = ["apple", "banana", "cherry", "date", "elderberry"]
long_words = [word.upper() for word in words if len(word) > 5]
print("Long words (uppercase):", long_words)
```

Nested list comprehensions

```
matrix2 = [[i * j for j in range(1, 4)] for i in range(1, 4)]
print("Multiplication table matrix:")
for row in matrix2:
    print(row)
```

Practical example - filtering and transforming

```
scores = [85, 92, 78, 65, 95, 88, 70]
grades = ['A' if s >= 90 else 'B' if s >= 80 else 'C' if s >= 70 else 'F' for s in scores]
print("Scores:", scores)
print("Grades:", grades)
```

Set comprehension

```
unique_lengths = {len(word) for word in words}
print("Unique word lengths:", unique_lengths)
```

Dictionary comprehension

```
square_dict = {x: x**2 for x in range(1, 6)}
print("Number squares dictionary:", square_dict)
```

Output

text

Traditional squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

List comprehension squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

Even numbers: [0, 2, 4, 6, 8, 10, 12, 14, 16, 18]

Even/odd classification: ['odd', 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even', 'odd', 'even']

Capitalized names: ['Alice', 'Bob', 'Charlie', 'David']

All pairs: [(1, 4), (1, 5), (1, 6), (2, 4), (2, 5), (2, 6), (3, 4), (3, 5), (3, 6)]

Flattened matrix: [1, 2, 3, 4, 5, 6, 7, 8, 9]

Long words (uppercase): ['BANANA', 'CHERRY', 'ELDERBERRY']

Multiplication table matrix:

[1, 2, 3]

[2, 4, 6]

[3, 6, 9]

Scores: [85, 92, 78, 65, 95, 88, 70]

Grades: ['B', 'A', 'C', 'F', 'A', 'B', 'C']

Unique word lengths: {5, 6, 7, 4}

Number squares dictionary: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

Syntax

text

[expression for item in iterable if condition]

Benefits

- More concise and readable
- Often faster than loops
- Great for data transformation
- Can include conditions

TOPIC 33: DECORATORS

Clear Explanation

Decorators are functions that modify the behavior of other functions. They allow you to add functionality to existing functions without changing their code.

Real-Time Example

Like adding a gift wrapper:

- Original: A gift (function)

- Decorator: Wrapping paper (adds functionality)
- Result: Wrapped gift (enhanced function)

Sample Code

python

Basic decorator

```
def my_decorator(func):  
    def wrapper():  
        print("Something is happening before the function")  
        func()  
        print("Something is happening after the function")  
    return wrapper
```

@my_decorator

```
def say_hello():  
    print("Hello!")
```

```
print("--- Basic decorator ---")  
say_hello()
```

Decorator with arguments

```
def timer(func):  
    import time  
    def wrapper(*args, **kwargs):  
        start = time.time()  
        result = func(*args, **kwargs)  
        end = time.time()  
        print(f"{func.__name__} took {end - start:.4f} seconds")  
        return result  
    return wrapper
```

@timer

```
def slow_function(seconds):
```

```
    import time
```

```
    time.sleep(seconds)
```

```
    return "Done!"
```

```
print("\n--- Timer decorator ---")
```

```
result = slow_function(1)
```

```
print(f"Result: {result}")
```

```
# Decorator with parameters
```

```
def repeat(n):
```

```
    def decorator(func):
```

```
        def wrapper(*args, **kwargs):
```

```
            results = []
```

```
            for _ in range(n):
```

```
                results.append(func(*args, **kwargs))
```

```
            return results
```

```
        return wrapper
```

```
    return decorator
```

```
@repeat(3)
```

```
def greet(name):
```

```
    return f"Hello, {name}!"
```

```
print("\n--- Repeat decorator ---")
```

```
greetings = greet("Alice")
```

```
print(greetings)
```

```
# Multiple decorators
```

```
def uppercase(func):
```

```
    def wrapper():
```

```
    result = func()
    return result.upper()
return wrapper
```

```
def exclamation(func):
    def wrapper():
        result = func()
        return result + "!!!"
    return wrapper
```

```
@uppercase
```

```
@exclamation
```

```
def get_message():
    return "hello world"
```

```
print("\n--- Multiple decorators ---")
print(get_message()) # Executes exclamation first, then uppercase
```

```
# Practical decorator - authentication
```

```
def requires_auth(func):
    def wrapper(user, *args, **kwargs):
        if user.get("authenticated", False):
            return func(user, *args, **kwargs)
        else:
            return "Access denied. Please log in."
    return wrapper
```

```
@requires_auth
```

```
def view_profile(user):
    return f"Profile: {user['name']}, Email: {user['email']}"
```

```

print("\n--- Authentication decorator ---")

user1 = {"name": "Alice", "email": "alice@example.com", "authenticated": True}
user2 = {"name": "Bob", "email": "bob@example.com", "authenticated": False}

print(view_profile(user1))
print(view_profile(user2))

```

Decorator with class

```

class CountCalls:

    def __init__(self, func):

        self.func = func

        self.count = 0

    def __call__(self, *args, **kwargs):

        self.count += 1

        print(f'Function {self.func.__name__} called {self.count} times')

        return self.func(*args, **kwargs)

```

@CountCalls

```

def test_function():

    return "Function executed"

```

```

print("\n--- Class decorator ---")

test_function()

test_function()

test_function()

```

Output

text

--- Basic decorator ---

Something is happening before the function

Hello!

Something is happening after the function

--- Timer decorator ---

slow_function took 1.0012 seconds

Result: Done!

--- Repeat decorator ---

['Hello, Alice!', 'Hello, Alice!', 'Hello, Alice!']

--- Multiple decorators ---

HELLO WORLD!!!

--- Authentication decorator ---

Profile: Alice, Email: alice@example.com

Access denied. Please log in.

--- Class decorator ---

Function test_function called 1 times

Function test_function called 2 times

Function test_function called 3 times

Common Use Cases

- Logging
- Timing functions
- Access control
- Caching
- Input validation

TOPIC 34: GENERATORS

Clear Explanation

Generators are functions that produce a sequence of results lazily (one at a time) instead of computing all at once. They use yield instead of return.

Real-Time Example

Like a streaming service vs downloading:

- Generator: Streams movie (one scene at a time)
- List: Downloads entire movie (all at once)
- More memory efficient for large data

Sample Code

python

Basic generator

```
def count_up_to(n):
```

```
    """Generate numbers from 1 to n"""
```

```
    i = 1
```

```
    while i <= n:
```

```
        yield i
```

```
        i += 1
```

```
print("--- Basic generator ---")
```

```
counter = count_up_to(5)
```

```
print(next(counter)) # 1
```

```
print(next(counter)) # 2
```

```
print(next(counter)) # 3
```

Using generator in loop

```
print("\n--- Using in loop ---")
```

```
for num in count_up_to(5):
```

```
    print(num, end=" ")
```

```
print()
```

Generator vs list memory comparison

```
import sys
```

```
def fibonacci_list(n):
```

```

"""Create list of Fibonacci numbers"""

fib = [0, 1]

for i in range(2, n):
    fib.append(fib[-1] + fib[-2])

return fib[:n]

def fibonacci_gen(n):
    """Generate Fibonacci numbers"""

    a, b = 0, 1

    for _ in range(n):
        yield a
        a, b = b, a + b

# Memory usage

list_fib = fibonacci_list(100000)
gen_fib = fibonacci_gen(100000)

print("\n--- Memory comparison ---")
print(f"List size: {sys.getsizeof(list_fib)} bytes")
print(f"Generator size: {sys.getsizeof(gen_fib)} bytes")

# Infinite generator

def infinite_numbers():
    """Generate infinite sequence of numbers"""

    n = 0

    while True:
        yield n
        n += 1

print("\n--- Infinite generator (first 10) ---")
numbers = infinite_numbers()

```

```
for _ in range(10):
    print(next(numbers), end=" ")
print("...")
```

Generator expression (like list comprehension)

```
print("\n--- Generator expression ---")
squares_gen = (x**2 for x in range(10))
print("Type:", type(squares_gen))
print("Squares:", list(squares_gen))
```

Practical example - reading large files

```
def read_large_file(file_path):
    """Read file line by line without loading entire file"""
    with open(file_path, 'r') as file:
        for line in file:
            yield line.strip()
```

Creating a sample file

```
with open("large_sample.txt", "w") as f:
    for i in range(100):
        f.write(f"This is line number {i}\n")
```

```
print("\n--- Reading large file (first 5 lines) ---")
for i, line in enumerate(read_large_file("large_sample.txt")):
    if i >= 5:
        break
    print(line)
```

Generator pipeline

```
def even_numbers(numbers):
    for n in numbers:
```

```

    if n % 2 == 0:
        yield n

def square(numbers):
    for n in numbers:
        yield n ** 2

def cube(numbers):
    for n in numbers:
        yield n ** 3

print("\n--- Generator pipeline ---")
numbers = range(10)
even = even_numbers(numbers)
squared = square(even)
cubed = cube(even_numbers(numbers))

print("Even numbers:", list(even))
print("Squares of evens:", list(squared))
print("Cubes of evens:", list(cubed))

```

Output

text

--- Basic generator ---

1

2

3

--- Using in loop ---

1 2 3 4 5

--- Memory comparison ---

List size: 800056 bytes

Generator size: 120 bytes

--- Infinite generator (first 10) ---

0 1 2 3 4 5 6 7 8 9 ...

--- Generator expression ---

Type: <class 'generator'>

Squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

--- Reading large file (first 5 lines) ---

This is line number 0

This is line number 1

This is line number 2

This is line number 3

This is line number 4

--- Generator pipeline ---

Even numbers: [0, 2, 4, 6, 8]

Squares of evens: [0, 4, 16, 36, 64]

Cubes of evens: [0, 8, 64, 216, 512]

Key Points

- yield returns value and pauses function
- Generator resumes where it left off
- Much more memory efficient for large datasets
- Can represent infinite sequences
- Can be used in pipelines

TOPIC 35: WORKING WITH DATES AND TIME

Clear Explanation

Python's datetime module provides classes for working with dates, times, and time intervals. It handles date arithmetic, formatting, and parsing.

Real-Time Example

Like a digital calendar:

- Track dates and events
- Calculate days until an event
- Format dates differently for different countries

Sample Code

```
python
```

```
from datetime import datetime, date, time, timedelta
```

```
import time as timer
```

```
# Current date and time
```

```
print("--- Current date and time ---")
```

```
now = datetime.now()
```

```
print("Current datetime:", now)
```

```
print("Current date:", now.date())
```

```
print("Current time:", now.time())
```

```
print("Year:", now.year)
```

```
print("Month:", now.month)
```

```
print("Day:", now.day)
```

```
print("Hour:", now.hour)
```

```
print("Minute:", now.minute)
```

```
print("Second:", now.second)
```

```
# Creating specific dates
```

```
print("\n--- Creating dates ---")
```

```
birthday = date(1995, 5, 15)
```

```
print("Birthday:", birthday)
```

```
meeting = datetime(2024, 12, 25, 14, 30, 0)
```

```
print("Meeting:", meeting)
```

Date arithmetic

```
print("\n--- Date arithmetic ---")
```

```
today = date.today()
```

```
future = today + timedelta(days=30)
```

```
past = today - timedelta(days=7)
```

```
print("Today:", today)
```

```
print("30 days from now:", future)
```

```
print("7 days ago:", past)
```

Time difference

```
birth = date(1990, 1, 1)
```

```
age = today - birth
```

```
print(f"Age in days: {age.days}")
```

```
print(f"Age in years: {age.days // 365}")
```

Formatting dates

```
print("\n--- Formatting dates ---")
```

```
now = datetime.now()
```

```
print("Default:", now)
```

```
print("YYYY-MM-DD:", now.strftime("%Y-%m-%d"))
```

```
print("MM/DD/YYYY:", now.strftime("%m/%d/%Y"))
```

```
print("Month name:", now.strftime("%B %d, %Y"))
```

```
print("Weekday:", now.strftime("%A"))
```

```
print("12-hour time:", now.strftime("%I:%M %p"))
```

```
print("Custom:", now.strftime("%A, %B %d, %Y at %I:%M %p"))
```

Parsing strings to dates

```
print("\n--- Parsing dates ---")
```

```
date_string = "2024-12-25"
parsed_date = datetime.strptime(date_string, "%Y-%m-%d")
print(f"Parsed: {parsed_date}")

date_string2 = "Dec 25, 2024"
parsed_date2 = datetime.strptime(date_string2, "%b %d, %Y")
print(f"Parsed: {parsed_date2}")
```

Working with time only

```
print("\n--- Time objects ---")
current_time = datetime.now().time()
print("Current time:", current_time)
```

```
alarm = time(7, 30, 0)
print("Alarm set for:", alarm)
```

Comparing dates

```
print("\n--- Comparing dates ---")
date1 = date(2024, 1, 1)
date2 = date(2024, 12, 31)

print(f"{date1} < {date2}: {date1 < date2}")
print(f"{date1} > {date2}: {date1 > date2}")
print(f"{date1} == {date2}: {date1 == date2}")
```

Timezone aware dates

```
print("\n--- Timezone (UTC) ---")
from datetime import timezone

utc_now = datetime.now(timezone.utc)
print("UTC now:", utc_now)
```

Practical example - countdown

```
def days_until_event(event_date):
    today = date.today()
    if event_date < today:
        return f"Event was { (today - event_date).days } days ago"
    else:
        return f"Event in { (event_date - today).days } days"

print("\n--- Countdown example ---")
new_year = date(2025, 1, 1)
print(f"New Year 2025: {days_until_event(new_year)}")
```

Timer example

```
print("\n--- Timer example ---")
print("Starting 3 second countdown...")
for i in range(3, 0, -1):
    print(i)
    timer.sleep(1)
print("Time's up!")
```

Output

text

--- Current date and time ---

Current datetime: 2024-01-15 14:30:25.123456

Current date: 2024-01-15

Current time: 14:30:25.123456

Year: 2024

Month: 1

Day: 15

Hour: 14

Minute: 30

Second: 25

--- Creating dates ---

Birthday: 1995-05-15

Meeting: 2024-12-25 14:30:00

--- Date arithmetic ---

Today: 2024-01-15

30 days from now: 2024-02-14

7 days ago: 2024-01-08

Age in days: 12428

Age in years: 34

--- Formatting dates ---

Default: 2024-01-15 14:30:25.123456

YYYY-MM-DD: 2024-01-15

MM/DD/YYYY: 01/15/2024

Month name: January 15, 2024

Weekday: Monday

12-hour time: 02:30 PM

Custom: Monday, January 15, 2024 at 02:30 PM

--- Parsing dates ---

Parsed: 2024-12-25 00:00:00

Parsed: 2024-12-25 00:00:00

--- Time objects ---

Current time: 14:30:25.123456

Alarm set for: 07:30:00

--- Comparing dates ---

2024-01-01 < 2024-12-31: True

2024-01-01 > 2024-12-31: False

2024-01-01 == 2024-12-31: False

--- Timezone (UTC) ---

UTC now: 2024-01-15 14:30:25.123456+00:00

--- Countdown example ---

New Year 2025: Event in 351 days

--- Timer example ---

Starting 3 second countdown...

3

2

1

Time's up!

Common Format Codes

Code	Meaning	Example
%Y	Year (4-digit)	2024
%y	Year (2-digit)	24
%m	Month (01-12)	01
%d	Day (01-31)	15
%H	Hour (00-23)	14
%I	Hour (01-12)	02
%M	Minute (00-59)	30

Code	Meaning	Example
%S	Second (00-59)	25
%p	AM/PM	PM
%A	Full weekday	Monday
%a	Short weekday	Mon
%B	Full month	January
%b	Short month	Jan

CONCLUSION

What You've Learned

This guide has covered the essential Python concepts:

Fundamentals

- Variables, data types, operators
- Strings, lists, tuples, sets, dictionaries
- Control flow (if, loops)

Functions & Modules

- Creating and using functions
- Lambda functions
- Importing modules

Object-Oriented Programming

- Classes and objects
- Inheritance
- Encapsulation, polymorphism

Advanced Concepts

- List comprehensions

- Generators
- Decorators
- Exception handling

File & Data Handling

- File operations
- JSON
- Regular expressions
- API requests

Development Tools

- Virtual environments
- Popular libraries